Artificial Intelligence & Machine Learning

Unit II: Informed Search Strategies and Game Playing

Study Notes with Mind Maps, Flowcharts, Diagrams and Cheat Sheet

B.C.A. Full Stack · School of Computer & Systems Sciences · Jaipur National University, Jaipur · Course code BCAFSSE501T23 (3 credits, 100 marks)

Contents

1. Unit 2 at a Glance
2. Informed Search and Best-First Search
3. The A* Algorithm
4. Heuristic Functions
5. Memory-Bounded Heuristic Search: IDA*, RBFS and SMA*
6. Game Playing and Optimal (Perfect) Decisions
7. Imperfect Decisions, Evaluation Functions and Alpha-Beta Pruning
8. Unit 2 Cheat Sheet
9. Important Questions
10. MCQ Practice
11. Glossary

Unit 2 at a Glance

Course: Artificial Intelligence & Machine Learning (BCAFSSE501T23) — B.C.A. Full Stack, 3 credits, 100 marks

Unit 2 syllabus: Informed Search Strategies — Best first search, A* algorithm, heuristic functions, Iterative deepening A* (IDA), small memory A* (SMA), Game playing — Perfect decision game, imperfect decision game, evaluation function, alpha-beta pruning.

Unit 1 taught you how an agent can search a state space *blindly* — breadth-first, depth-first, uniform-cost and iterative deepening search. These methods know nothing about where the goal lies, so they waste effort exploring hopeless directions. Unit 2 is about making search *intelligent*. We give the agent a "sense of direction" called a **heuristic**, and we study the algorithms that use it: best-first search, greedy search, A*, and the memory-saving variants IDA* and SMA*. In the second half of the unit, the search problem gets an opponent. In games such as chess and tic-tac-toe, the agent must plan against an adversary who is trying to make it lose. We study how to make perfect decisions with minimax, how to make good decisions under time limits with an evaluation function, and how alpha-beta pruning lets a program look twice as deep for the same effort.

Syllabus Coverage Map

Every phrase of the official syllabus is mapped below to the chapter where it is taught. Use this table to make sure you have revised each item before the exam.

Table 2.1: Syllabus phrase to chapter mapping

Syllabus phrase	Where it is covered
Informed search strategies	Informed Search and Best-First Search (informed vs uninformed, heuristic idea)
Best first search	Informed Search and Best-First Search (framework, greedy best-first, full trace)
A* algorithm	The A* Algorithm (pseudocode, Romania trace, admissibility, consistency, optimality)
Heuristic functions	Heuristic Functions (8-puzzle h1 and h2, effective branching factor, dominance, relaxed problems)
Iterative deepening A* (IDA)	Memory-Bounded Heuristic Search (IDA* algorithm, contours, full trace)
Small memory A* (SMA)	Memory-Bounded Heuristic Search (SMA* algorithm, forgetting the worst leaf, worked example)
Game playing	Game Playing and Optimal Decisions (games as search, game tree, game types)
Perfect decision game	Game Playing and Optimal Decisions (minimax to terminal states; perfect-information games)
Imperfect decision game	Imperfect Decisions, Evaluation Functions and Alpha-Beta Pruning (cutoff, imperfect-information and chance games)
Evaluation function	Imperfect Decisions, Evaluation Functions and Alpha-Beta Pruning (weighted linear functions, chess, tic-tac-toe)
Alpha-beta pruning	Imperfect Decisions, Evaluation Functions and Alpha-Beta Pruning (algorithm, worked trees, move ordering)

Learning Outcomes

After studying this unit you should be able to:

- Distinguish informed (heuristic) search from uninformed (blind) search and state the role of the evaluation function f(n).
- Explain the general best-first search framework with OPEN and CLOSED lists, and trace greedy best-first search on a map.
- Write the A* algorithm, compute f(n) = g(n) + h(n) for every node, and trace A* by hand to find an optimal path.
- Define admissible and consistent (monotonic) heuristics and explain why they guarantee optimality.
- Compute the misplaced-tiles (h1) and Manhattan-distance (h2) heuristics for an 8-puzzle state, and explain effective branching factor, dominance and relaxed problems.
- Describe and trace the memory-bounded algorithms IDA*, RBFS and SMA*, and compare all informed searches on completeness, optimality, time and space.
- Model a two-player game as a search problem and draw its game tree.
- Apply the minimax algorithm to back up values in a game tree (perfect decisions).
- Explain imperfect real-time decisions: cutoff test, evaluation function, horizon effect and quiescence search.
- Apply alpha-beta pruning step by step, identify pruned branches, and explain the effect of move ordering.

How to Use These Notes

- **Read in order.** Each chapter builds on the previous one. A* needs best-first search; IDA* and SMA* need A*; alpha-beta needs minimax.
- **Redo every worked example with a pencil.** The traces (greedy, A*, IDA*, SMA*, minimax, alpha-beta) are the most frequently asked 10-mark questions. All numerical traces in these notes have been checked by running the algorithms.
- **Use the callout boxes.** "Key Point" boxes hold definitions you must reproduce exactly; "Exam Tip" boxes tell you what examiners look for; "Memory Trick" boxes give mnemonics; "Worked Example" boxes show complete solutions.
- **Revise with the back-matter.** The Cheat Sheet compresses the unit into a few pages for the night before the exam. The Important Questions, MCQ Practice and Glossary help you test yourself.

> **Exam Tip:** In this unit, examiners love numerical problems: "Apply A* to the given graph", "Find the minimax value and show the branches pruned by alpha-beta", "Compute h1 and h2 for the 8-puzzle state". Always show a table of OPEN/CLOSED lists or a tree with values at every node — marks are given for the steps, not just the final answer.

Unit Mind Map

The mind map below shows the seven big ideas of the unit and the key terms attached to each. Try redrawing it from memory after you finish the unit.

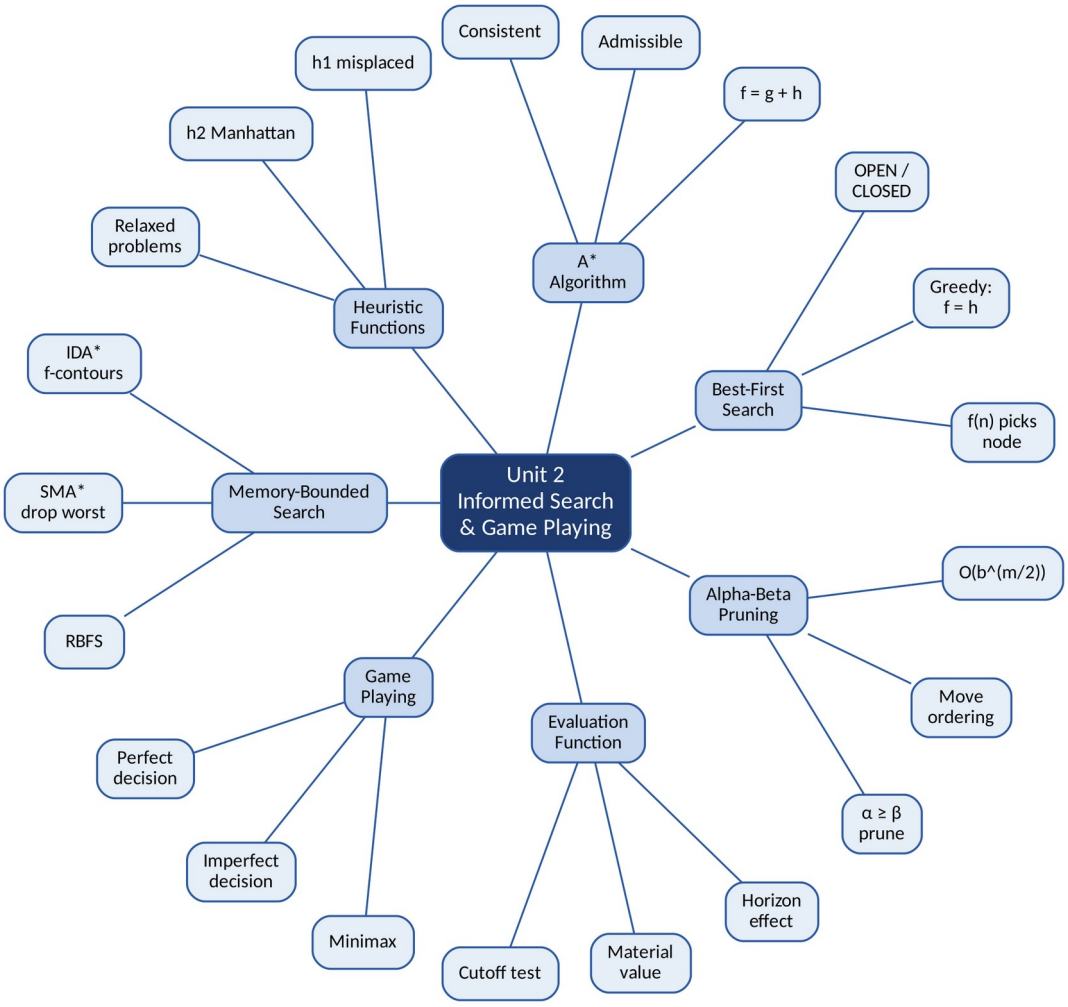


Figure 2.1: Unit 2 mind map — informed search and game playing

Where This Unit Fits

Table 2.2: How Unit 2 connects to the rest of the course

Unit 1 (before)	Unit 2 (this unit)	Later units
Problem formulation, state space	Heuristics guide the same state space	Knowledge representation and reasoning
Blind search: BFS, DFS, UCS, IDDFS	Informed search: greedy, A*, IDA*, SMA*	Machine learning can *learn* heuristics
Single agent, no opponent	Adversarial search: minimax, alpha-beta	Planning and decision making under uncertainty

Informed Search and Best-First Search

Why Blind Search Is Not Enough

In Unit 1 we solved problems by searching a **state space**: a set of states connected by actions, with an initial state and a goal test. The uninformed strategies — breadth-first search (BFS), uniform-cost search (UCS), depth-first search (DFS), depth-limited search and iterative deepening DFS (IDDFS) — all share one weakness. They generate successors in a fixed order and have *no idea* whether a particular state is close to the goal or far away from it.

Consider driving from Arad to Bucharest in Romania. A blind algorithm such as BFS will happily explore towns to the west and north of Arad, even though Bucharest lies to the south-east. A human driver would never do this, because the human has extra knowledge: "Bucharest is roughly in *that* direction, so roads heading that way look more promising." This extra problem-specific knowledge, beyond the bare problem definition, is what makes search **informed**.

The cost of ignoring such knowledge is huge. If the branching factor is b and the solution lies at depth d, BFS generates on the order of b^d nodes. For the 8-puzzle with b ≈ 3 and a typical solution depth of 22, that is about 3^22 ≈ 3.1 × 10^10 nodes. A good heuristic can cut this to a few thousand.

Informed (Heuristic) Search

Informed search (also called **heuristic search**) is a search strategy that uses problem-specific knowledge, in the form of a heuristic function, to decide which node to expand next. The aim is to find a solution while expanding far fewer nodes than a blind strategy.

The Heuristic Function h(n)

A **heuristic function**, written **h(n)**, is the *estimated* cost of the cheapest path from the state at node n to a goal state.

- h(n) depends only on the *state* at node n, not on how we reached it.
- For any goal node, h(goal) = 0.
- h(n) is an *estimate*, not the true value. The true cost of the cheapest path from n to the goal is written **h*(n)**.
- A heuristic is usually cheap to compute — far cheaper than actually solving the remaining problem.

Typical examples are:

- **Route finding:** straight-line distance (as the crow flies) from the current city to the destination, written h_SLD(n).
- **8-puzzle:** number of tiles out of place, or the sum of distances of tiles from their goal squares.
- **Robot navigation on a grid:** Manhattan distance |x1 − x2| + |y1 − y2| to the target cell.

> **Key Point:** A heuristic function h(n) estimates the cost from node n to the nearest goal. It is the only component of informed search that carries domain knowledge — the search algorithm itself stays general. Better heuristics (closer to the true cost h*(n) without exceeding it) make search faster.

The Word "Heuristic"

The word comes from the Greek *heuriskein*, "to find or discover" (the same root as "Eureka"). In AI, a heuristic is a rule of thumb: it usually helps, but it gives no absolute guarantee by itself. Whether a heuristic

search is *guaranteed* to find the best solution depends on both the algorithm and the properties of h, as we shall see with A*.

Informed vs Uninformed Search

Table 2.3: Informed vs uninformed search

Basis	Uninformed (blind) search	Informed (heuristic) search
Knowledge used	Only the problem definition	Problem definition plus heuristic h(n)
Direction of search	No sense of direction to goal	Guided towards promising states
Node selection	Fixed rule (FIFO, LIFO, lowest g)	Lowest evaluation f(n) involving h(n)
Efficiency	Usually expands very many nodes	Usually expands far fewer nodes
Examples	BFS, DFS, UCS, DLS, IDDFS	Greedy best-first, A*, IDA*, RBFS, SMA*, AO*
Optimality	BFS/UCS/IDDFS optimal under conditions	A* optimal if h admissible/consistent
Cost of each step	Very cheap	Extra time to compute h(n)
Also called	Blind or brute-force search	Heuristic or directed search

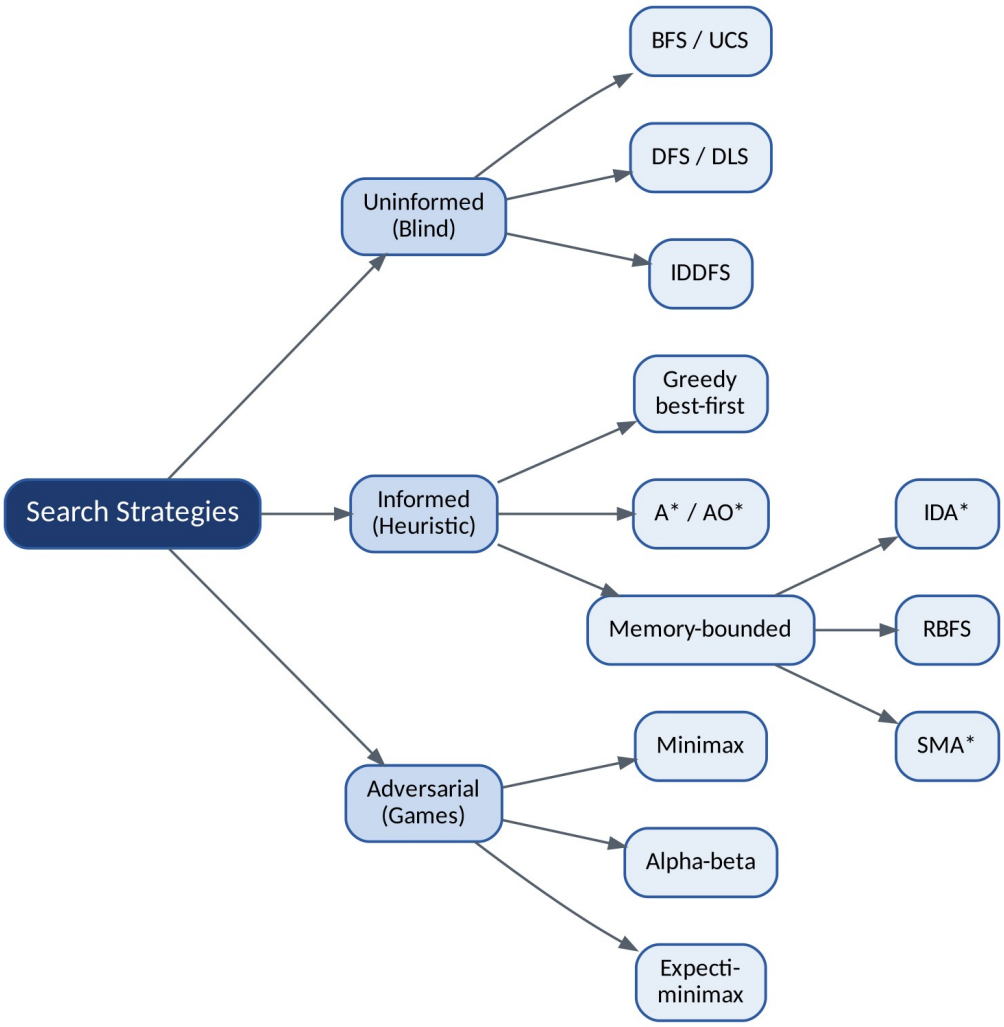


Figure 2.2: Classification of search strategies studied in Units 1 and 2

Exam Tip: "Differentiate informed and uninformed search" is a standard 5-mark question. Write at least six points in a table (knowledge used, node selection, efficiency, examples, optimality, cost per step) and finish with one example each, such as BFS vs A*.

Best-First Search: The General Framework

Best-first search is a general search strategy in which the node selected for expansion is the one with the *best* (lowest) value of an **evaluation function f(n)**. It is an instance of the general graph-search algorithm where the frontier is a **priority queue** ordered by f.

The name is slightly misleading. If we truly knew which node was "best", we would not need to search at all. Best-first search expands the node that *appears* best according to f.

Different choices of f(n) give different algorithms:

Table 2.4: One framework, many algorithms — the effect of f(n)

Choice of f(n)	Resulting algorithm
f(n) = g(n) (path cost so far)	Uniform-cost search (blind)
f(n) = h(n) (estimate to goal)	Greedy best-first search
f(n) = g(n) + h(n)	A* search
f(n) = depth(n)	Breadth-first search (special case)
f(n) = −depth(n)	Depth-first search (special case)

Here **g(n)** is the actual cost of the path from the start node to n, and h(n) is the heuristic estimate from n to the goal.

OPEN and CLOSED Lists

Classical textbooks (Rich and Knight, Nilsson) describe best-first search using two lists:

- **OPEN** — nodes that have been generated but not yet expanded (the **frontier** or **fringe**). It is kept as a priority queue sorted by f.
- **CLOSED** — nodes that have already been expanded (the **explored set**). It prevents us from expanding the same state again and from looping in cycles.

Best-First Search Algorithm

```
function BEST-FIRST-SEARCH(problem, f):
    OPEN   <- priority queue containing START, keyed by f(START)
    CLOSED <- empty set
    loop:
        if OPEN is empty: return FAILURE
        n <- node in OPEN with LOWEST f(n); remove it from OPEN
        if GOAL-TEST(n): return path from START to n
        add n to CLOSED
        for each successor m of n:
            compute f(m)
            if m not in OPEN and not in CLOSED:
                insert m into OPEN with parent n
            else if m in OPEN with a higher f:
                replace that entry by m (cheaper path found)
```

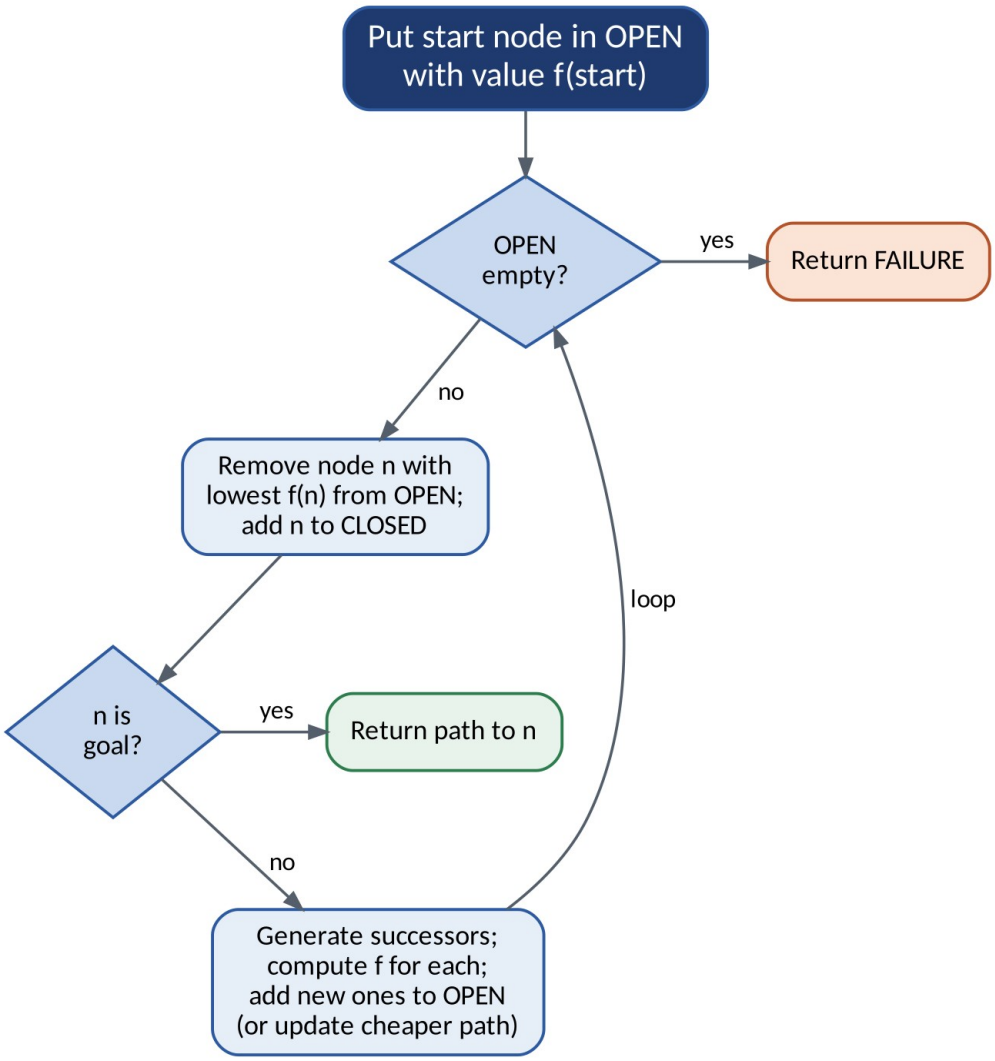


Figure 2.3: Flowchart of the general best-first search loop

Two details deserve attention:

1. **The goal test is applied when a node is selected for expansion**, not when it is generated. This matters for A*: a goal node may be generated through a poor path first, and a better path may be found before it is selected.
2. **Duplicate handling.** If a state reappears with a better f-value, the cheaper path replaces the old one. For A* with a consistent heuristic, a state on CLOSED never needs to be reopened.

> **Memory Trick:** "OPEN is the To-Do list, CLOSED is the Done list." Best-first always picks the To-Do item with the smallest f. Change the formula for f and you change the algorithm: g gives UCS, h gives Greedy, g + h gives A* — "G, H, G-plus-H: Uniform, Greedy, A-star."

The Romania Route-Finding Problem

We use the classic example from Russell and Norvig throughout this unit. A tourist is in **Arad** and must reach **Bucharest**. The map below is a subset of the Romanian road map. Each edge is labelled with the road distance in kilometres. Each node also shows h_SLD, the straight-line distance from that city to Bucharest.

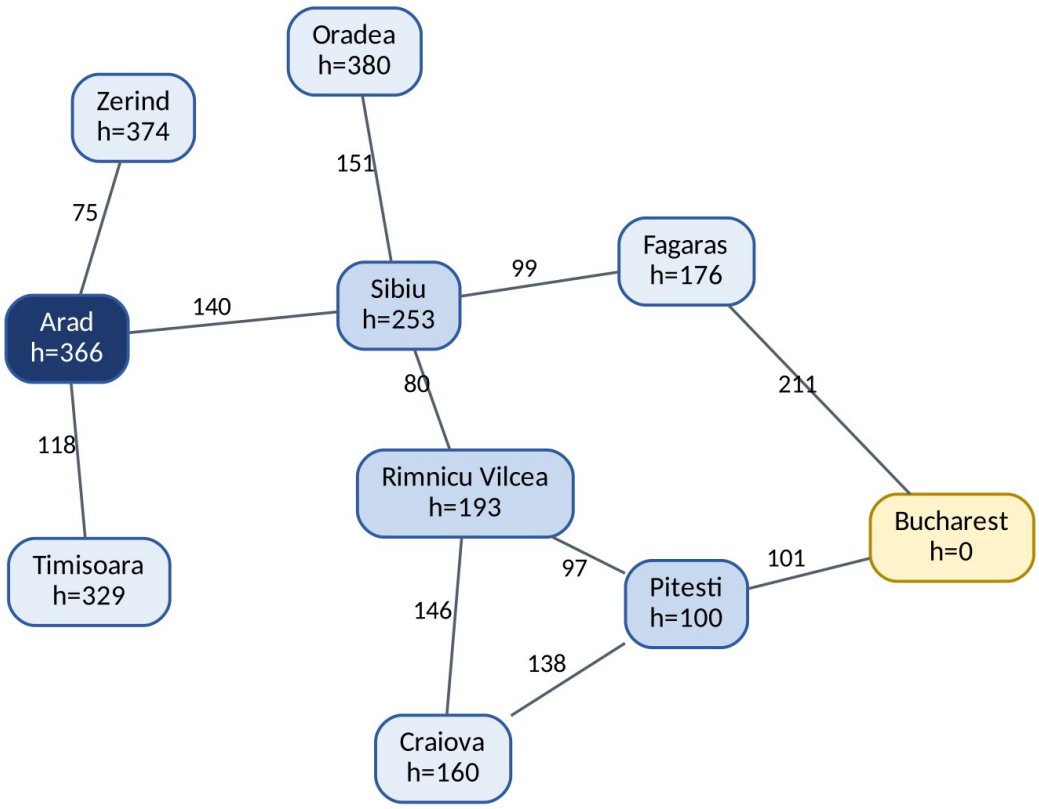


Figure 2.4: Romania map subset — road distances on edges, straight-line distance to Bucharest (h) in nodes

Table 2.5: Straight-line distances to Bucharest (h_SLD)

City	h_SLD to Bucharest	City	h_SLD to Bucharest
Arad	366	Pitesti	100
Bucharest	0	Rimnicu Vilcea	193
Craiova	160	Sibiu	253
Fagaras	176	Timisoara	329
Oradea	380	Zerind	374

Table 2.6: Road distances in the Romania subset

Road	km	Road	km
Arad – Sibiu	140	Sibiu – Oradea	151
Arad – Timisoara	118	Fagaras – Bucharest	211
Arad – Zerind	75	Rimnicu Vilcea – Pitesti	97
Sibiu – Fagaras	99	Rimnicu Vilcea – Craiova	146
Sibiu – Rimnicu Vilcea	80	Pitesti – Bucharest	101
Craiova – Pitesti	138		

Straight-line distance is a sensible heuristic because the shortest road between two cities can never be shorter than the straight line joining them. Notice that h_SLD cannot be computed from the problem

description alone — it needs extra knowledge (the coordinates of the cities). That extra knowledge is exactly what makes the search *informed*.

Greedy Best-First Search

Greedy best-first search expands the node that appears closest to the goal, using only the heuristic:

f(n) = h(n)

It is "greedy" because at every step it tries to get as close to the goal as it can, without considering how much it has already spent (g is ignored).

Greedy Algorithm

```
function GREEDY-BEST-FIRST(problem):
    return BEST-FIRST-SEARCH(problem, f = h)
```

Full Worked Trace: Arad to Bucharest

We run greedy best-first search with OPEN sorted by h and a CLOSED list (graph search). When a node is expanded, its successors that are already on OPEN or CLOSED are not added again.

Worked Example — Greedy best-first search from Arad to Bucharest: Start at Arad (h = 366). Expanding Arad gives Sibiu (253), Timisoara (329) and Zerind (374); Sibiu has the lowest h. Expanding Sibiu gives Fagaras (176), Oradea (380) and Rimnicu Vilcea (193); Arad is already CLOSED. Fagaras now has the lowest h. Expanding Fagaras gives Bucharest (0). Bucharest is then selected and passes the goal test. Path: Arad → Sibiu → Fagaras → Bucharest, cost 140 + 99 + 211 = 450 km.

Table 2.7: Greedy best-first search trace (OPEN/CLOSED)

Step	Node expanded (h)	OPEN after expansion (sorted by h)	CLOSED
0	—	Arad 366	—
1	Arad 366	Sibiu 253, Timisoara 329, Zerind 374	Arad
2	Sibiu 253	Fagaras 176, Rimnicu V. 193, Timisoara 329, Zerind 374, Oradea 380	Arad, Sibiu
3	Fagaras 176	Bucharest 0, Rimnicu V. 193, Timisoara 329, Zerind 374, Oradea 380	Arad, Sibiu, Fagaras
4	Bucharest 0 — GOAL	Rimnicu V. 193, Timisoara 329, Zerind 374, Oradea 380	Arad, Sibiu, Fagaras, Bucharest

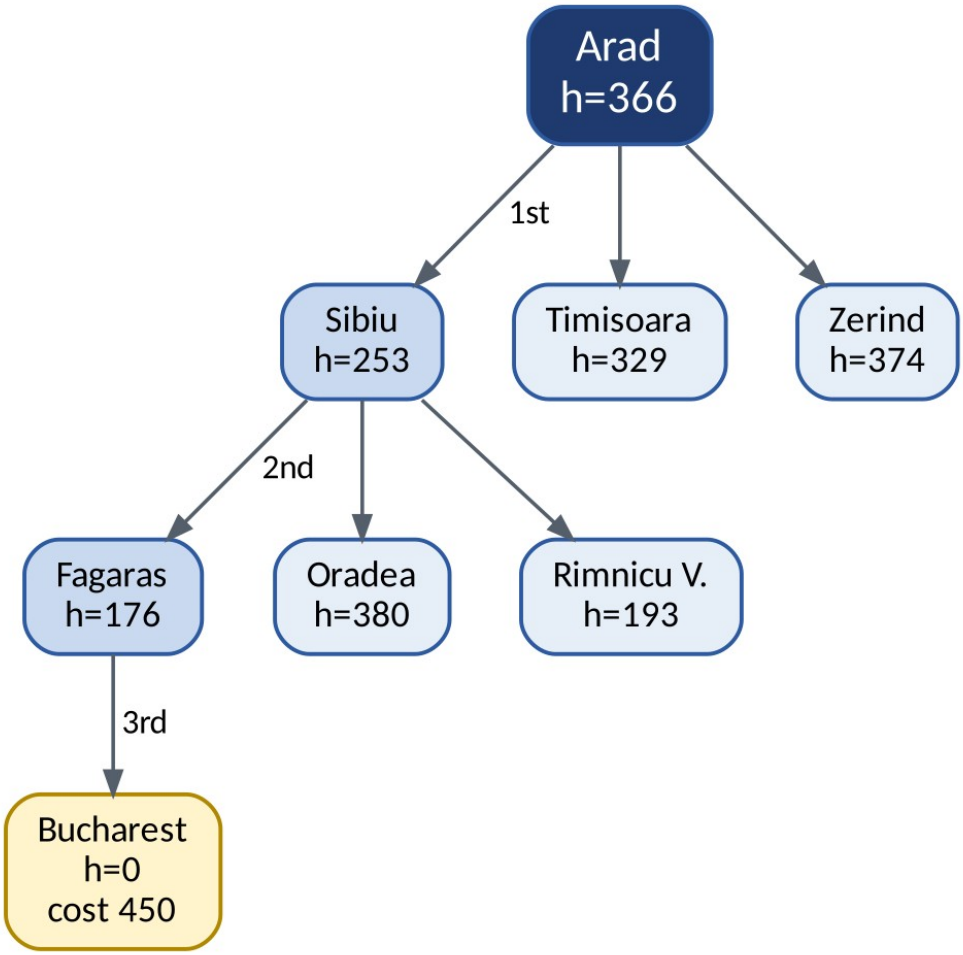


Figure 2.5: Greedy best-first search tree from Arad; numbers on edges show expansion order

Analysing the Result

Greedy search found a solution after expanding only three nodes (Arad, Sibiu, Fagaras) before reaching the goal — that is very efficient. But the solution is **not optimal**:

- Greedy path: Arad → Sibiu → Fagaras → Bucharest = 140 + 99 + 211 = **450 km**.
- Optimal path: Arad → Sibiu → Rimnicu Vilcea → Pitesti → Bucharest = 140 + 80 + 97 + 101 = **418 km**.

Greedy chose Fagaras because Fagaras *looks* closer (176 < 193), ignoring the fact that the road from Fagaras to Bucharest is very long (211 km). The path through Rimnicu Vilcea and Pitesti is 32 km shorter.

Greedy Search Can Be Incomplete

In the full Romania map, consider travelling from Iasi to Fagaras. The heuristic suggests going to Neamt first, because Neamt is closest to Fagaras by straight line — but Neamt is a dead end. Tree-search greedy will go back to Iasi, then Neamt again, looping forever. The graph-search version with a CLOSED list avoids this loop and is complete in finite state spaces, but tree-search greedy is not complete even in finite spaces. In infinite spaces, greedy search can follow an endless path whose h keeps decreasing without reaching the goal.

Properties of Greedy Best-First Search

Table 2.8: Properties of greedy best-first search

Property	Greedy best-first search
Complete?	No (tree search can loop); yes for graph search in finite spaces
Optimal?	No — e.g. found 450 km instead of 418 km
Time complexity	O(b^m) in the worst case, m = maximum depth
Space complexity	O(b^m) — keeps all nodes in memory
Practical speed	Often very fast with a good heuristic

The worst case is bad, but a good heuristic usually gives a dramatic reduction in practice. Greedy search resembles depth-first search in the way it prefers to follow one path all the way to the goal, and it backs up when it hits a dead end.

> **Key Point:** Greedy best-first search uses f(n) = h(n). It is fast but neither optimal nor (in tree-search form) complete, because it ignores the cost already paid, g(n). A* fixes this by adding g(n) back in.

Advantages and Disadvantages of Greedy Search

Advantages

- Simple to implement — just a priority queue on h.
- Often finds a solution quickly while expanding few nodes.
- Useful when *any* reasonable solution is acceptable and speed matters more than optimality.

Disadvantages

- Not optimal: may return an expensive path.
- Can be misled by a heuristic into dead ends (the Iasi–Neamt problem).
- Worst-case time and space are exponential.

Real-World Uses of Best-First Search

- **GPS and map navigation:** variants of best-first search with geographic heuristics compute routes in road networks.
- **Video-game path-finding:** non-player characters use greedy or A*-style searches on grids or navigation meshes.
- **Puzzle solving:** sliding-tile puzzles and Rubik's-cube solvers use heuristics such as Manhattan distance and pattern databases.
- **Planning systems:** automated planners such as heuristic-search planners use relaxed-problem heuristics to guide plan search.

Summary

- Informed search uses a heuristic h(n), an estimate of the cost from n to the goal, to guide expansion; uninformed search does not.
- Best-first search always expands the node with the lowest evaluation function f(n), keeping an OPEN priority queue and a CLOSED set.

- The choice of f determines the algorithm: f = g gives uniform-cost search, f = h gives greedy best-first search, f = g + h gives A*.
- On the Romania problem with straight-line distance, greedy search reaches Bucharest via Fagaras with cost 450 — fast but not optimal (the optimum is 418).
- Greedy search is not optimal, is incomplete as a tree search, and has worst-case time and space O(b^m).

The A* Algorithm

Idea of A*

A* search (pronounced "A-star") is the most widely known form of best-first search. It was introduced by Peter Hart, Nils Nilsson and Bertram Raphael in 1968. A* combines the strengths of two algorithms:

- **Uniform-cost search** uses g(n), the cost already paid. It is optimal but slow because it has no sense of direction.
- **Greedy best-first search** uses h(n), the estimated cost still to go. It is fast but not optimal.

A* evaluates each node by adding the two:

f(n) = g(n) + h(n)

where

- **g(n)** = actual cost of the path from the start node to n (known exactly);
- **h(n)** = estimated cost of the cheapest path from n to a goal (heuristic);
- **f(n)** = estimated cost of the cheapest solution that passes through n.

A* always expands the node on OPEN with the lowest f(n). Intuitively, it asks: "If I go through this node, what is my best guess of the total trip cost?" — and it explores the node whose total looks cheapest.

> **Key Point:** In A*, f(n) = g(n) + h(n) is the estimated total cost of the cheapest solution through node n. Provided h(n) never overestimates (admissible), A* is both complete and optimal.

A* Pseudocode

```
function A-STAR(start, goal-test, h):
    g[start] <- 0
    f[start] <- g[start] + h(start)
    OPEN   <- priority queue {start} ordered by f
    CLOSED <- empty set
    while OPEN is not empty:
        n <- OPEN.pop-lowest-f()
        if goal-test(n): return PATH(n), g[n]
        add n to CLOSED
        for each successor m of n with step cost c(n, m):
            new-g <- g[n] + c(n, m)
            if m in CLOSED and new-g >= g[m]: continue
            if m not in OPEN or new-g < g[m]:
                g[m] <- new-g
                f[m] <- g[m] + h(m)
                parent[m] <- n
                if m in CLOSED: move m back to OPEN   (needed only
                                  if h is admissible but inconsistent)
                else: add / update m in OPEN
    return FAILURE
```

Points to note in the pseudocode:

1. The goal test is done when a node is **removed** from OPEN, not when it is generated. This is essential for optimality.
2. When a cheaper path to a node already on OPEN is found, its g, f and parent are updated.
3. With a **consistent** heuristic, the first time a node is expanded its g-value is already optimal, so CLOSED nodes never need reopening.

Full Worked Trace: Arad to Bucharest

We use the same map and straight-line-distance heuristic as in the previous chapter (Figure 2.4, Tables 2.5 and 2.6).

Step-by-Step Computation

Step 0. OPEN = {Arad: g = 0, h = 366, f = 366}.

Step 1. Expand Arad (f = 366). Successors:

- Sibiu: g = 140, f = 140 + 253 = 393
- Timisoara: g = 118, f = 118 + 329 = 447
- Zerind: g = 75, f = 75 + 374 = 449

Step 2. Expand Sibiu (f = 393). Successors (Arad is already CLOSED with a cheaper g, so it is ignored; its f would have been 280 + 366 = 646):

- Fagaras: g = 140 + 99 = 239, f = 239 + 176 = 415
- Oradea: g = 140 + 151 = 291, f = 291 + 380 = 671
- Rimnicu Vilcea: g = 140 + 80 = 220, f = 220 + 193 = 413

Step 3. Expand Rimnicu Vilcea (f = 413). Successors (Sibiu is CLOSED; its f via this path would be 300 + 253 = 553):

- Craiova: g = 220 + 146 = 366, f = 366 + 160 = 526
- Pitesti: g = 220 + 97 = 317, f = 317 + 100 = 417

Step 4. Expand Fagaras (f = 415). Successors (Sibiu CLOSED):

- Bucharest: g = 239 + 211 = 450, f = 450 + 0 = 450

Bucharest is now on OPEN, but A* does **not** stop: the goal test is applied only when a node is selected for expansion, and Pitesti (417) is still cheaper than 450.

Step 5. Expand Pitesti (f = 417). Successors (Rimnicu Vilcea CLOSED):

- Bucharest: g = 317 + 101 = 418, f = 418 + 0 = 418. This is cheaper than the 450 entry, so Bucharest's g is updated to 418 and its parent becomes Pitesti.
- Craiova: g = 317 + 138 = 455, f = 615. Craiova is already on OPEN with g = 366, so this worse path is ignored.

Step 6. Select Bucharest (f = 418). Goal test succeeds. Stop.

OPEN/CLOSED Table

Table 2.9: A trace from Arad to Bucharest (f = g + h)*

Step	Expanded (f)	OPEN after expansion, sorted by f	CLOSED
0	—	Arad 366	—
1	Arad 366	Sibiu 393, Timisoara 447, Zerind 449	Arad
2	Sibiu 393	Rimnicu V. 413, Fagaras 415, Timisoara 447, Zerind 449, Oradea 671	Arad, Sibiu
3	Rimnicu V. 413	Fagaras 415, Pitesti 417, Timisoara 447, Zerind 449, Craiova 526, Oradea 671	+ Rimnicu V.

Step	Expanded (f)	OPEN after expansion, sorted by f	CLOSED
4	Fagaras 415	Pitesti 417, Timisoara 447, Zerind 449, Bucharest 450, Craiova 526, Oradea 671	+ Fagaras
5	Pitesti 417	Bucharest 418, Timisoara 447, Zerind 449, Craiova 526, Oradea 671	+ Pitesti
6	Bucharest 418 — GOAL	(stop)	+ Bucharest

g, h and f for Every Generated Node

Table 2.10: Node evaluations in the A trace*

Node (parent)	g(n)	h(n)	f(n) = g + h
Arad (start)	0	366	366
Sibiu (Arad)	140	253	393
Timisoara (Arad)	118	329	447
Zerind (Arad)	75	374	449
Rimnicu Vilcea (Sibiu)	220	193	413
Fagaras (Sibiu)	239	176	415
Oradea (Sibiu)	291	380	671
Pitesti (Rimnicu V.)	317	100	417
Craiova (Rimnicu V.)	366	160	526
Bucharest (Fagaras)	450	0	450 — later replaced
Bucharest (Pitesti)	418	0	418 — selected, goal

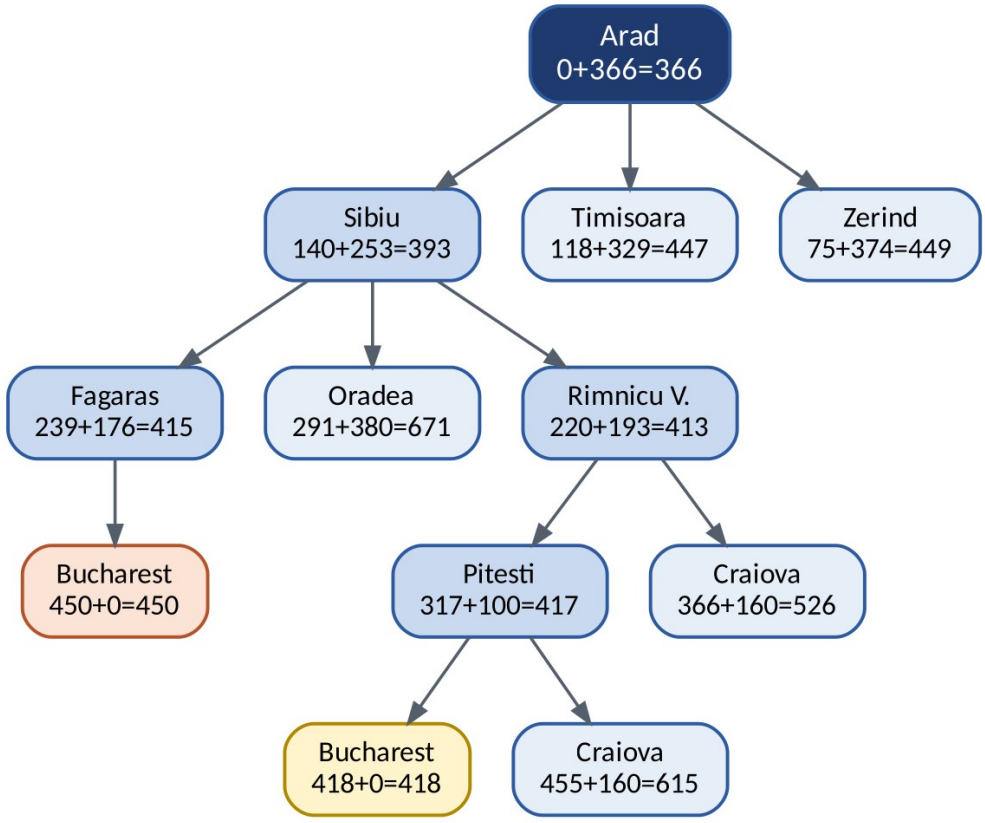


Figure 2.6: A search tree with f = g + h at every node; the yellow node is the goal reached through the optimal path*

Worked Example — Result of A* on the Romania map: A* returns Arad → Sibiu → Rimnicu Vilcea → Pitesti → Bucharest with cost 140 + 80 + 97 + 101 = 418 km, which is the optimal route (greedy search returned 450 km). This trace was verified by running a Python implementation. Note that along the solution path the f-values never decrease: 366, 393, 413, 417, 418 — a sign that the heuristic is consistent.

Exam Tip: In an A* numerical, always (1) write g, h and f for every generated node, (2) show OPEN sorted by f after each step, (3) point out that the goal is tested on selection, not generation — here Bucharest appears with f = 450 but is selected only with f = 418. Missing this point is the most common error and costs marks.

Admissible Heuristics

A heuristic h is **admissible** if it never overestimates the true cost to reach the goal:

$h(n) \le h^*(n)$ for every node n

where $h^*(n)$ is the true cost of the optimal path from n to a goal. An admissible heuristic is **optimistic**: it thinks the problem is easier (or equal) to what it really is.

Why h_SLD Is Admissible

A straight line is the shortest distance between two points, so no road route can be shorter than the straight-line distance. Hence $h_{SLD}(n) \le h^*(n)$ for every city.

Table 2.11: Checking admissibility of h_SLD on the Romania subset

City	h_SLD(n)	True road cost h*(n)	h ≤ h*?
Arad	366	418	Yes
Sibiu	253	278	Yes
Rimnicu Vilcea	193	198	Yes
Fagaras	176	211	Yes
Pitesti	100	101	Yes
Craiova	160	239	Yes

What Goes Wrong With an Overestimating Heuristic

Take a small graph: S–A costs 1, A–G costs 5 (total 6); S–B costs 2, B–G costs 2 (total 4, optimal). Let h(S) = 0, h(A) = 1, h(G) = 0, but h(B) = 6 — an overestimate, because the true cost from B is only 2.

- Expand S: A gets f = 1 + 1 = 2, B gets f = 2 + 6 = 8.
- Expand A: G gets f = 6 + 0 = 6.
- OPEN = {G: 6, B: 8}. G is selected and A* returns cost **6**, missing the optimal cost **4**.

The overestimate made B look worse than it is, so A* never explored it. This is why admissibility is required.

Consistent (Monotonic) Heuristics

A heuristic is **consistent** (also called **monotonic**) if, for every node n and every successor n′ generated by action a with step cost c(n, a, n′):

$$h(n) \le c(n, a, n') + h(n')$$

and h(goal) = 0. This is a form of the **triangle inequality**: the estimate from n cannot be more than the cost of stepping to n′ plus the estimate from n′.

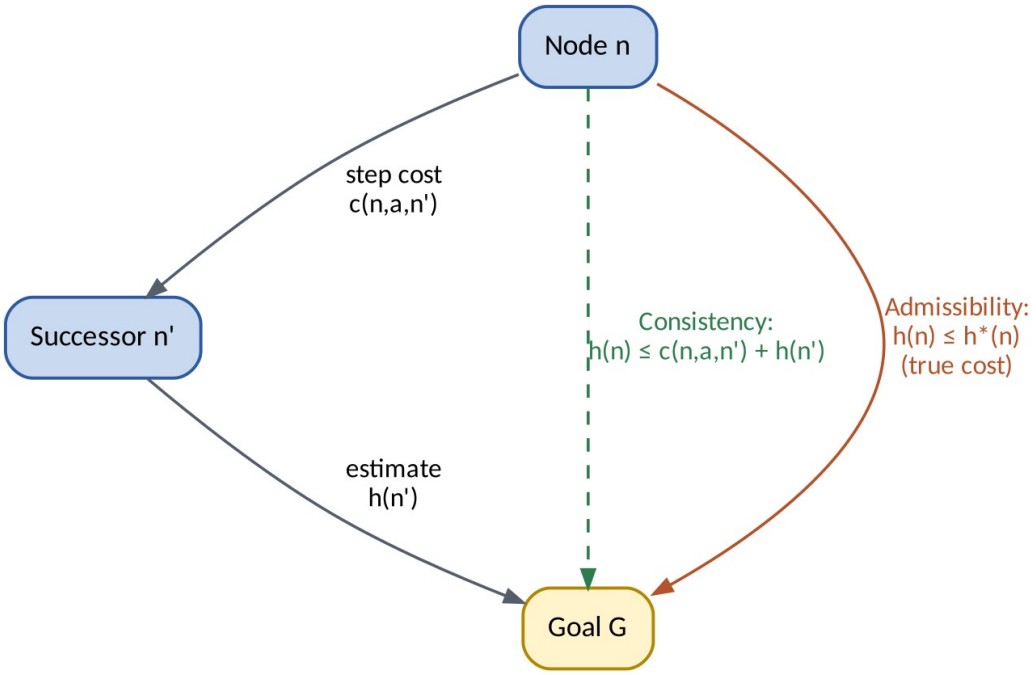


Figure 2.7: Admissibility (h(n) ≤ true cost h(n)) and consistency (triangle inequality between n, n′ and the goal)*

Checking Consistency on the Romania Subset

For each road we check h(n) ≤ c + h(n′) in both directions. For example:

- Rimnicu Vilcea → Pitesti: 193 ≤ 97 + 100 = 197. Holds.
- Pitesti → Bucharest: 100 ≤ 101 + 0 = 101. Holds.
- Sibiu → Rimnicu Vilcea: 253 ≤ 80 + 193 = 273. Holds.
- Fagaras → Bucharest: 176 ≤ 211 + 0 = 211. Holds.

Every edge of the subset satisfies the inequality, so h_SLD is consistent here (and straight-line distance is consistent in general, because of the geometric triangle inequality).

Consequences of Consistency

1. **Consistent ⇒ admissible.** By induction from the goal (where h = 0): if every step satisfies the triangle inequality, the estimate can never exceed the total true cost. The converse is not always true, though admissible-but-inconsistent heuristics are rare in practice.
2. **f is non-decreasing along any path.** If n′ is a successor of n, then f(n′) = g(n) + c + h(n′) ≥ g(n) + h(n) = f(n).
3. **First expansion is optimal.** Whenever A* (graph search) selects a node for expansion, the best path to that node has already been found. So CLOSED nodes never need to be reopened.

Table 2.12: Admissible vs consistent heuristics

Property	Admissible	Consistent (monotonic)
Definition	h(n) ≤ h*(n)	h(n) ≤ c(n, a, n′) + h(n′)
Compares h with	True total cost to goal	One step plus neighbour's estimate
Relationship	Weaker condition	Stronger: consistent ⇒ admissible
A* tree search optimal?	Yes	Yes
A* graph search optimal?	Only if CLOSED nodes may be reopened	Yes, no reopening needed
f along a path	May decrease	Never decreases

> **Memory Trick:** "Admissible = Always optimistic; Consistent = Climbs steadily." An admissible h never says the trip is longer than it is; a consistent h makes f climb steadily (never drop) as you walk along a path.

Optimality of A*: The Proof Idea

Theorem. If h is admissible, A* tree search is optimal. If h is consistent, A* graph search is optimal.

Proof sketch (admissible case). Let C* be the cost of an optimal solution, and suppose A* is about to select a suboptimal goal G2 with g(G2) > C*.

1. Because G2 is a goal, h(G2) = 0, so f(G2) = g(G2) > C*.
2. Consider an optimal solution path. Some node n on that path must still be on OPEN (the start was on OPEN, and the path has not been fully expanded, otherwise the optimal goal would have been found).
3. Because h is admissible, f(n) = g(n) + h(n) ≤ g(n) + h*(n) = C*.
4. So f(n) ≤ C* < f(G2). A* always picks the lowest f, so it would expand n before G2.

Repeating the argument, every node on the optimal path is expanded before G2, so the optimal goal is selected first. Contradiction — hence A* never returns a suboptimal goal.

f-Contours

With a consistent heuristic, the f-values of expanded nodes form a non-decreasing sequence. We can draw **contours** on the state space, like contour lines on a hill map: contour 400 encloses all states with f ≤ 400, contour 420 encloses those with f ≤ 420, and so on. A* expands nodes in bands of increasing f.

- A* expands **all** nodes with f(n) < C*.
- A* may expand **some** nodes with f(n) = C*.
- A* expands **no** node with f(n) > C*.

In our Romania trace C* = 418. The nodes expanded before the goal — Arad 366, Sibiu 393, Rimnicu Vilcea 413, Fagaras 415, Pitesti 417 — all have f < 418. Timisoara (447) and Zerind (449) were never expanded: they were **pruned** without any explicit test, because their f exceeded C*.

With uniform-cost search (h = 0) the contours are circles around the start. With a good heuristic, the contours stretch towards the goal and become narrow, so fewer nodes are expanded.

Optimal Efficiency

A* is **optimally efficient** for a given consistent heuristic: no other optimal algorithm that extends search paths from the root using the same heuristic is guaranteed to expand fewer nodes than A* (apart from tie-breaking among nodes with f = C*). Any such algorithm that skipped a node with f(n) < C* would risk missing the optimal solution.

Properties of A*

Table 2.13: Properties of A search*

Property	A* search
Complete?	Yes, if branching factor is finite and every step cost ≥ some ε > 0
Optimal?	Yes — tree search if h admissible; graph search if h consistent
Optimally efficient?	Yes, for a given consistent heuristic
Time complexity	Exponential in the worst case; depends on heuristic error
Space complexity	Exponential — keeps all generated nodes in memory
Main weakness	Runs out of memory long before running out of time

The time taken depends on the **heuristic error**. If the absolute error $|h^*(n) - h(n)|$ grows no faster than the logarithm of the true cost, A* runs in polynomial time — but for most real problems the error is at least proportional to the path cost, so A* is still exponential. Its biggest practical problem is **memory**: it keeps every generated node on OPEN or CLOSED. This motivates the memory-bounded algorithms IDA* and SMA* in a later chapter.

> **Key Point:** A* is complete, optimal and optimally efficient with a consistent heuristic, but it stores every generated node. "A* usually runs out of space long before it runs out of time" — this sentence from Russell and Norvig is worth quoting in exams.

Special Cases of A*

*Table 2.14: Special cases and variants of A**

If we set	A* becomes	Comment
h(n) = 0 for all n	Uniform-cost search	Optimal but uninformed
g(n) = 0 (ignore g)	Greedy best-first search	Fast, not optimal
h(n) = h*(n) (perfect)	Expands only nodes on the optimal path	Ideal but unrealistic
h overestimates	Weighted / inadmissible A*	Faster, may lose optimality
f = g + W·h, W > 1	Weighted A*	Solution cost at most W × optimal

Advantages and Disadvantages of A*

Advantages

- Optimal and complete under mild, well-understood conditions.
- Optimally efficient — no other comparable algorithm does better with the same heuristic.
- Flexible: any admissible heuristic can be plugged in; better heuristics give faster search.
- Widely used in practice: map routing, game path-finding, robot motion planning, puzzle solving, natural-language parsing.

Disadvantages

- Memory hungry: stores all generated nodes, which is impractical for large problems.
- Performance depends heavily on the quality of the heuristic.
- Still exponential time for most hard problems.
- Ties and real-valued costs can cause many nodes with nearly equal f to be expanded.

AO* Algorithm (Brief)

Some problems are best solved by **problem decomposition**: a goal is broken into sub-goals, *all* of which must be solved (AND), or into alternatives, *any one* of which is enough (OR). Such problems are represented by an **AND-OR graph**. For example, "Get a degree" may be achieved by "Pass all semesters" (AND of several sub-goals) or by "Complete a lateral-entry route" (an OR alternative).

AO* is the best-first search algorithm for AND-OR graphs:

- It maintains a partial **solution graph** rather than a single path.
- It expands the most promising unexpanded node of the current best partial solution.
- After expansion it **revises costs upwards** (cost propagation): the cost of an AND node is the sum of its children's costs plus arc costs; the cost of an OR node is the minimum over its alternatives.
- Nodes found to be unsolvable are marked FUTILE.
- With an admissible heuristic, AO* finds an optimal solution graph.

Table 2.15: A vs AO**

Feature	A*	AO*
Graph type	OR graph (state space)	AND-OR graph (problem reduction)
Solution is	A path	A sub-graph (solution tree)

Feature	A*	AO*
Cost of node	g + h	Propagated from children (sum for AND, min for OR)
Typical use	Route finding, puzzles	Problem decomposition, theorem proving, planning

Summary

- A* evaluates nodes by f(n) = g(n) + h(n), the estimated cost of the cheapest solution through n.
- On the Romania map, A* finds Arad → Sibiu → Rimnicu Vilcea → Pitesti → Bucharest, cost 418, the optimal route; Bucharest first appears with f = 450 but the goal test is applied only on selection.
- Admissible: h(n) ≤ h*(n). Consistent: h(n) ≤ c(n, a, n′) + h(n′). Consistency implies admissibility and makes f non-decreasing along paths.
- A* is complete, optimal (tree search with admissible h, graph search with consistent h) and optimally efficient, but its memory use is exponential.
- AO* extends best-first search to AND-OR graphs for problem decomposition.

Heuristic Functions

What Makes a Good Heuristic?

We have seen that the performance of A* depends almost entirely on the heuristic h(n). A good heuristic should be:

1. **Admissible** (never overestimate) — so that A* stays optimal.
2. **Consistent** — so that graph-search A* never needs to reopen nodes.
3. **Accurate** — as close as possible to the true cost h*(n), so that few nodes are expanded.
4. **Cheap to compute** — evaluating h is done for every generated node, so an expensive heuristic can cancel out its benefits.

There is a trade-off between points 3 and 4. The perfect heuristic h = h* would make A* walk straight to the goal, but computing h* exactly is as hard as solving the problem itself.

The 8-Puzzle Problem

The **8-puzzle** consists of a 3 × 3 board with eight numbered tiles and one blank space. A tile adjacent to the blank can slide into it. The goal is to reach a specified goal configuration.

Some facts about the 8-puzzle:

- Actions: move the blank Up, Down, Left or Right (where possible).
- Branching factor: about 3 on average (2 in a corner, 3 on an edge, 4 in the centre).
- Average solution cost for a random instance: about 22 steps.
- Number of reachable states: 9!/2 = 181,440. (Only half of all 9! arrangements can be reached from a given state.)
- For the 15-puzzle (4 × 4), the reachable states number 16!/2 ≈ 1.05 × 10^13, so a good heuristic becomes essential.

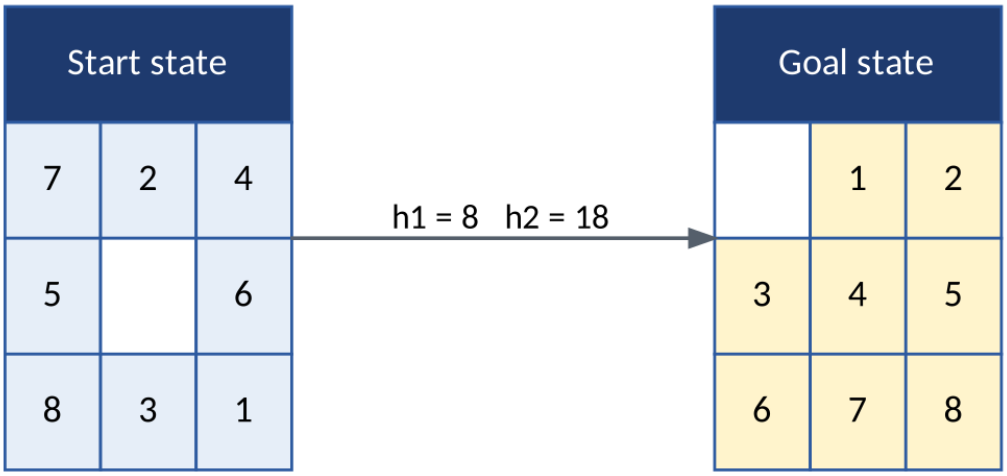


Figure 2.8: An 8-puzzle start state and goal state (Russell and Norvig); h1 = 8 and h2 = 18 for this start state

The start state in Figure 2.8 (rows: 7 2 4 / 5 _ 6 / 8 3 1) needs **26 moves** in an optimal solution, so h*(start) = 26.

Two Classic Heuristics for the 8-Puzzle

h1: Number of Misplaced Tiles

h1(n) = the number of tiles that are not in their goal position (the blank is not counted).

h1 is admissible because every misplaced tile must be moved at least once, so at least h1 moves are needed.

h2: Manhattan (City-Block) Distance

h2(n) = the sum, over all tiles, of the horizontal plus vertical distance of each tile from its goal square. It is called the **Manhattan distance** or **city-block distance** because, like a taxi in Manhattan, a tile can only move along rows and columns, never diagonally.

For a tile at (row r, column c) whose goal is (r′, c′), its distance is $|r - r'| + |c - c'|$.

h2 is admissible because each move can bring only one tile one step closer to its goal square.

Worked Numeric Example (Figure 2.8)

Number rows and columns 0, 1, 2 from the top-left. Start: 7 2 4 / 5 _ 6 / 8 3 1. Goal: _ 1 2 / 3 4 5 / 6 7 8.

Table 2.16: Computing h1 and h2 for the start state of Figure 2.8

Tile	Start (row, col)	Goal (row, col)	Misplaced?	Manhattan distance
1	(2, 2)	(0, 1)	Yes	2 + 1 = 3
2	(0, 1)	(0, 2)	Yes	0 + 1 = 1
3	(2, 1)	(1, 0)	Yes	1 + 1 = 2
4	(0, 2)	(1, 1)	Yes	1 + 1 = 2
5	(1, 0)	(1, 2)	Yes	0 + 2 = 2
6	(1, 2)	(2, 0)	Yes	1 + 2 = 3
7	(0, 0)	(2, 1)	Yes	2 + 1 = 3
8	(2, 0)	(2, 2)	Yes	0 + 2 = 2
Total			h1 = 8	h2 = 18

So h1 = 8 and h2 = 3 + 1 + 2 + 2 + 2 + 3 + 3 + 2 = 18. Both are less than the true cost of 26, as admissibility requires, and h2 is much closer to the truth than h1. (These values were checked with a short Python program.)

Worked Example — A second 8-puzzle with A*: Start: 2 8 3 / 1 6 4 / 7 _ 5; goal: 1 2 3 / 8 _ 4 / 7 6 5. Misplaced tiles are 2, 8, 1 and 6, so h1 = 4. Manhattan distances are 2→1, 8→2, 1→1, 6→1, so h2 = 5. Running A* (verified in Python) gives the optimal 5-move solution: move the blank Up, Up, Left, Down, Right. In our run A* with h1 expanded 7 nodes, and A* with h2 expanded only 6 nodes, because h2 equals the true cost at the start and guides the search straight down the solution path.

The first two levels of the A* search with h1 for this second example:

Table 2.17: First two levels of A with h1 on the second 8-puzzle example*

Node (blank move)	Board	g	h1	f = g + h1
Start	283 / 164 / 7_5	0	4	4
Up	283 / 1_4 / 765	1	3	4

Node (blank move)	Board	g	h1	f = g + h1
Left	283 / 164 / _75	1	5	6
Right	283 / 164 / 75_	1	5	6
Up, Up	2_3 / 184 / 765	2	3	5
Up, Left	283 / _14 / 765	2	3	5
Up, Right	283 / 14_ / 765	2	4	6

The search continues from the f = 5 nodes: Up, Up, Left gives _23 / 184 / 765 (g 3, h1 2, f 5), then Down gives 123 / _84 / 765 (g 4, h1 1, f 5), then Right gives the goal 123 / 8_4 / 765 (g 5, h1 0, f 5).

> **Exam Tip:** When computing h1 and h2, never count the blank. Draw a small table with one row per tile (as in Table 2.16) — it makes your working easy to check and earns full method marks even if one value is wrong.

Effective Branching Factor

How do we measure the quality of a heuristic? One way is the **effective branching factor b***. If A* generates a total of N nodes to find a solution at depth d, then b* is the branching factor that a *uniform* tree of depth d would need in order to contain N + 1 nodes:

N + 1 = 1 + b* + (b*)^2 + … + (b*)^d

- A smaller b* means a better heuristic. The ideal is b* close to 1, meaning the search goes almost straight to the goal.
- b* tends to be fairly constant across problem instances of different sizes, so it can be measured on small test problems and used to predict behaviour on larger ones.

> **Worked Example — Effective branching factor:** If A* finds a solution at depth d = 5 using N = 52 nodes, then we solve 1 + b* + (b*)^2 + (b*)^3 + (b*)^4 + (b*)^5 = 53. Trying b* = 1.9 gives about 51.5; trying b* = 1.92 gives about 53.1. So b* ≈ 1.92 (solved numerically: 1.917).

Comparing h1 and h2 Experimentally

Russell and Norvig report average results over 100 random 8-puzzle instances for each solution depth. At depth 12, for example:

Table 2.18: Effect of the heuristic on 8-puzzle search cost (Russell and Norvig, depth 12)

Search method	Nodes generated (d = 12)	Effective branching factor
Iterative deepening search (no heuristic)	3,644,035	2.78
A* with h1 (misplaced tiles)	227	1.42
A* with h2 (Manhattan distance)	73	1.24

The improvement from blind search to heuristic search is several orders of magnitude, and h2 is consistently better than h1.

Dominance

For any node n of the 8-puzzle, h2(n) ≥ h1(n). (A misplaced tile contributes exactly 1 to h1 and at least 1 to h2.) We say that **h2 dominates h1**.

Formally, if h1 and h2 are both admissible and h2(n) ≥ h1(n) for every node n, then h2 **dominates** h1.

Why dominance matters: A* expands every node with f(n) < C*, i.e. every node with h(n) < C* − g(n). Since h2 ≥ h1, every node that A* with h2 expands will also be expanded by A* with h1 (h1 may expand more). Therefore a dominating heuristic never expands more nodes (apart from tie-breaking).

Rule: It is always better to use a heuristic with higher values, provided it is admissible and its computation time is not too large.

Combining Heuristics: The Max Rule

If we have several admissible heuristics h1, h2, ..., hm and none clearly dominates the others, we can combine them:

h(n) = max { h1(n), h2(n), ..., hm(n) }

This composite heuristic is admissible (each component is admissible, so the maximum is too), it is consistent if all components are consistent, and it dominates every component.

> **Key Point:** If h2(n) ≥ h1(n) for all n and both are admissible, h2 dominates h1 and A* with h2 will never expand more nodes than A* with h1. The maximum of several admissible heuristics is admissible and dominates each of them.

Inventing Heuristics: Relaxed Problems

Where do heuristics such as h1 and h2 come from? A powerful method is to build a **relaxed problem** — a version of the problem with fewer restrictions on the actions.

Key fact: The cost of an optimal solution to a relaxed problem is an admissible heuristic for the original problem. This is because every solution of the original problem is also a solution of the relaxed one, so the relaxed optimum can only be cheaper or equal. Moreover, such a heuristic is also consistent, because it is an exact cost for the relaxed problem and therefore obeys the triangle inequality.

Relaxations of the 8-Puzzle

The 8-puzzle rule, written formally, is:

*A tile can move from square A to square B if A is horizontally or vertically adjacent to B **and** B is blank.*

Removing one or both conditions gives three relaxed problems:

Table 2.19: Deriving 8-puzzle heuristics from relaxed problems

Relaxed rule	Exact solution cost of relaxed problem	Resulting heuristic
(a) A tile can move from A to B if A is adjacent to B	Each tile moves straight to its goal by its distance	h2 — Manhattan distance
(b) A tile can move from A to B if B is blank	Each misplaced tile needs about one swap with the blank	Gaschnig's heuristic
(c) A tile can move from A to B (no conditions)	Each misplaced tile jumps home in one move	h1 — misplaced tiles

Because relaxed problems are written in a formal language, programs can generate them automatically. The **ABSOLVER** program (Prieditis, 1993) generated heuristics this way and found a new heuristic for the 8-puzzle better than any previous one, and the first useful heuristic for Rubik's cube.

Relaxation in Route Finding

For route finding, removing the restriction "you must travel along roads" gives the relaxed problem of flying in a straight line. Its exact cost is the straight-line distance, h_SLD — the heuristic we used on the Romania map.

> **Memory Trick:** "Relax the rules, get a heuristic." Remove *adjacent* and *blank* → teleport tiles → h1 (misplaced). Remove only *blank* → slide through other tiles → h2 (Manhattan). Remove *roads* → fly → straight-line distance.

Pattern Databases and Subproblems

Another source of admissible heuristics is the cost of solving a **subproblem**. For the 8-puzzle, consider only tiles 1, 2, 3 and 4 (treating the others as indistinguishable "don't care" tiles). The exact cost of getting these four tiles into place is a lower bound on the cost of the whole problem.

A **pattern database** stores these exact subproblem costs for every possible configuration of the chosen tiles. It is built once, by searching backwards from the goal, and then looked up during search. Lookups for different subproblems (for example, tiles 1–4 and tiles 5–8) can be combined with the max rule. If we count only the moves of the subproblem's own tiles, the costs of disjoint subproblems can even be *added* — these are **disjoint pattern databases**, which have made random 15-puzzle instances solvable in milliseconds and helped solve 24-puzzle instances.

Learning Heuristics From Experience

A heuristic can also be **learned**. The agent solves many instances of the problem (for example, many 8-puzzles) and records, for each state on a solution path, its actual cost to the goal. These (state, cost) pairs form training data for a machine learning algorithm, which learns a function that predicts the cost from a new state.

Learning usually works from **features** of a state rather than the raw state. For the 8-puzzle, useful features are:

- x1(n) = number of misplaced tiles;
- x2(n) = number of pairs of adjacent tiles that are not adjacent in the goal state.

A simple approach is a linear combination:

$h(n) = c1 \cdot x1(n) + c2 \cdot x2(n)$

where the constants c1 and c2 are adjusted to fit the training data (for example, by least-squares regression). Neural networks, decision trees and reinforcement learning methods can also be used. Learned heuristics may be very accurate, but they are **not guaranteed to be admissible or consistent**, so A* using them may lose optimality.

Table 2.20: Ways of obtaining heuristic functions

Method	Admissible guaranteed?	Main idea
Relaxed problem	Yes (and consistent)	Exact cost with fewer restrictions
Subproblem / pattern database	Yes	Exact cost for a subset of the problem
Max of admissible heuristics	Yes	Take the largest lower bound
Learning from experience	No	Predict cost from features using ML
Domain expert's rule of thumb	Not necessarily	Hand-crafted estimate

Heuristics in Other Domains

Table 2.21: Heuristics in common problem domains

Problem	Typical heuristic	Admissible?
Road route finding	Straight-line distance to goal	Yes
Grid path-finding (4-way moves)	Manhattan distance	Yes
Grid path-finding (8-way moves)	Chebyshev or octile distance	Yes
8-puzzle / 15-puzzle	Misplaced tiles, Manhattan, pattern databases	Yes
Travelling salesperson	Cost of minimum spanning tree of unvisited cities	Yes
Blocks-world planning	Number of blocks out of place	Depends on action costs

Summary

- A good heuristic is admissible, consistent, accurate and cheap to compute.
- 8-puzzle heuristics: h1 = number of misplaced tiles; h2 = sum of Manhattan distances. For the start 7 2 4 / 5 _ 6 / 8 3 1, h1 = 8, h2 = 18, true cost 26.
- The effective branching factor b* measures heuristic quality: N + 1 = 1 + b* + (b*)^2 + … + (b*)^d; closer to 1 is better.
- h2 dominates h1 (h2 ≥ h1 everywhere), so A* with h2 never expands more nodes. The max of admissible heuristics is admissible and dominant.
- Admissible heuristics come from relaxed problems, pattern databases (subproblems) and, without guarantees, from learning on features such as h(n) = c1·x1(n) + c2·x2(n).

Memory-Bounded Heuristic Search: IDA*, RBFS and SMA*

The Memory Problem of A*

A* keeps every generated node in memory — the frontier (OPEN) and the explored set (CLOSED). For problems with large state spaces, the number of stored nodes grows exponentially with depth. On a typical computer of the 1990s, A* on hard 15-puzzle instances would exhaust memory within minutes. Even today, memory, not time, is the usual limit on A*.

The solution is to trade time for space. Three classical **memory-bounded** variants of A* are:

- **IDA*** (Iterative Deepening A*) — uses depth-first search with an f-cost limit;
- **RBFS** (Recursive Best-First Search) — mimics best-first search in linear space;
- **SMA*** (Simplified Memory-Bounded A*) — uses all available memory and forgets the worst nodes when memory is full.

The idea is borrowed from uninformed search: just as iterative deepening DFS gives BFS-like results with DFS-like memory, IDA* gives A*-like results with DFS-like memory.

Iterative Deepening A* (IDA*)

IDA* (Korf, 1985) is iterative deepening search in which the cutoff is not the *depth* but the **f-cost** f(n) = g(n) + h(n).

How IDA* Works

1. Set the initial **threshold** (cutoff) to f(start) = h(start).
2. Perform a depth-first search from the start. Whenever a node with f(n) > threshold is generated, do not expand it; just remember its f-value.
3. If a goal is found with f ≤ threshold, return it.
4. Otherwise, set the new threshold to the **smallest f-value that exceeded the old threshold** in this iteration, and repeat from step 2.

Each iteration explores all nodes inside one **f-contour**, and the next iteration moves the contour outwards just enough to include at least one new node.

IDA* Pseudocode

```
function IDA-STAR(start):
    threshold <- h(start)
    loop:
        t <- DFS-CONTOUR(start, g = 0, threshold, path = [start])
        if t = FOUND: return path
        if t = INFINITY: return FAILURE
        threshold <- t              // smallest f that exceeded the limit

function DFS-CONTOUR(n, g, threshold, path):
    f <- g + h(n)
    if f > threshold: return f        // cut off; report f
    if GOAL-TEST(n): return FOUND
    min <- INFINITY
    for each successor m of n not already on path:
        t <- DFS-CONTOUR(m, g + c(n, m), threshold, path + [m])
        if t = FOUND: return FOUND
```

```
        if t < min: min <- t
    return min
```

Full Worked Trace: IDA* on the Romania Map

Using the same map and h_SLD as before (successors taken in alphabetical order; cities already on the current path are skipped to avoid cycles):

Table 2.22: IDA iterations on the Romania problem*

Iteration	Threshold	Nodes expanded (in order)	f-values cut off	Next threshold
1	366	Arad	Sibiu 393, Timisoara 447, Zerind 449	393
2	393	Arad, Sibiu	Fagaras 415, Oradea 671, Rimnicu V. 413, Timisoara 447, Zerind 449	413
3	413	Arad, Sibiu, Rimnicu V.	Fagaras 415, Craiova 526, Pitesti 417, others	415
4	415	Arad, Sibiu, Fagaras, Rimnicu V.	Bucharest 450 (via Fagaras), Pitesti 417, others	417
5	417	Arad, Sibiu, Fagaras, Rimnicu V., Pitesti	Bucharest 418 (via Pitesti), Bucharest 450, others	418
6	418	Arad, Sibiu, Fagaras, Rimnicu V., Pitesti	—	Goal found

In iteration 6 the threshold is 418. The depth-first search explores Sibiu's children in order: Fagaras (415) is expanded, but Bucharest via Fagaras has f = 450 > 418 and is cut off. Rimnicu Vilcea (413) is expanded, then Pitesti (417), and Bucharest via Pitesti has f = 418 ≤ 418 — it is the goal. IDA* returns Arad → Sibiu → Rimnicu Vilcea → Pitesti → Bucharest, cost 418, the same optimal answer as A*. (This trace was verified with a Python implementation.)

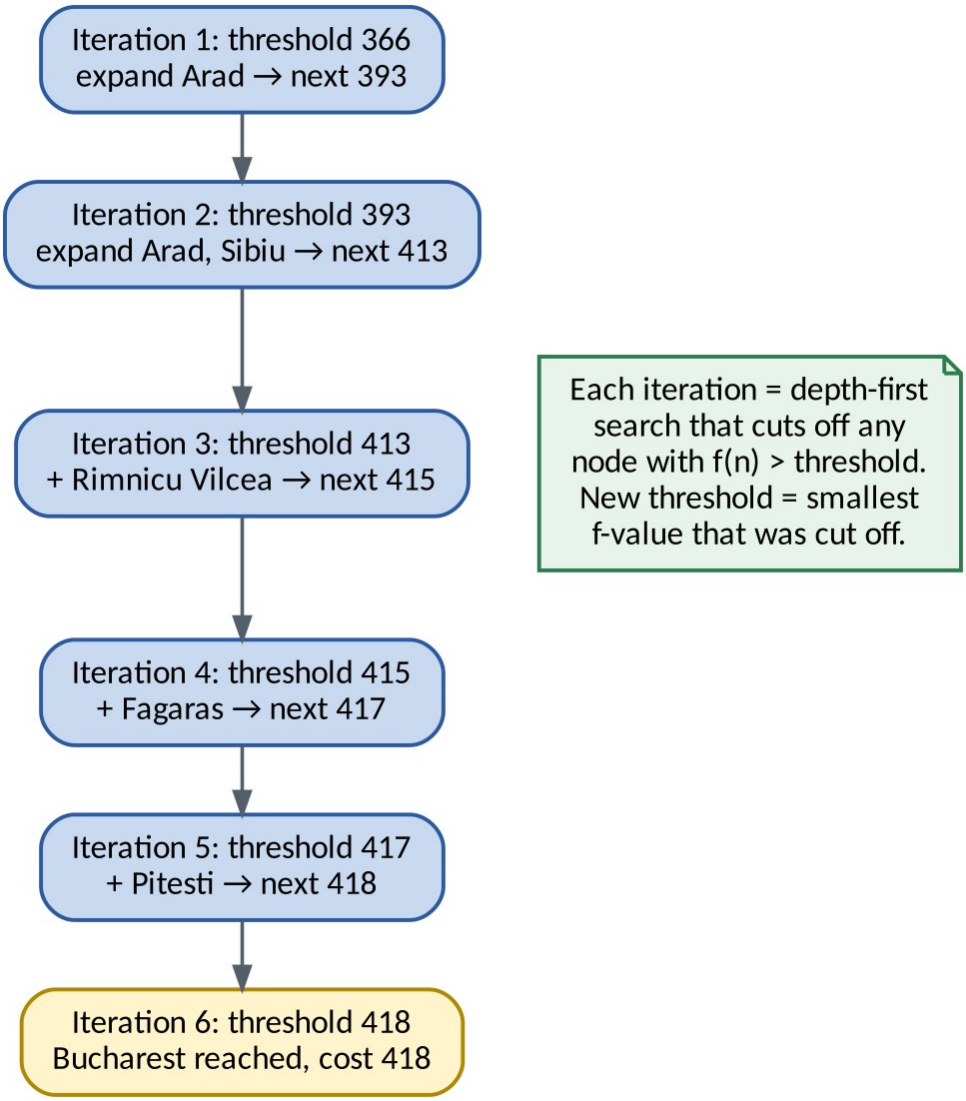


Figure 2.9: IDA iterations — each iteration searches one f-contour and raises the threshold to the smallest f-value that was cut off*

Worked Example — Why IDA* re-expands nodes: In the trace, Arad is expanded in all six iterations and Sibiu in five. This repetition is the price of using memory proportional only to the depth of the current path. Because the number of nodes usually grows exponentially with f, the final iteration dominates the total work, just as the last level dominates in iterative deepening DFS.

Properties of IDA*

*Table 2.23: Properties of IDA**

Property	IDA*
Complete?	Yes (finite b, step costs ≥ ε > 0)
Optimal?	Yes, if h is admissible
Space	O(b·d) — linear, only the current path and siblings
Time	Exponential; may repeat much work across iterations

Property	IDA*
Best suited for	Unit step costs, integer f-values (e.g. 8- and 15-puzzles)
Weak point	Real-valued costs: each iteration may add only one new node

Strengths. IDA* is simple, has tiny memory requirements and no priority-queue overhead. It was the first algorithm to optimally solve random instances of the 15-puzzle.

Weaknesses. If f-values are all different (real-valued step costs), each new threshold may add just one node, so the number of iterations can approach the number of nodes and the total time can become quadratic in the number of nodes A* would expand. IDA* also forgets everything between iterations and cannot detect repeated states other than those on the current path.

> **Key Point:** IDA* = iterative deepening with an f-cost cutoff instead of a depth cutoff. Start with threshold h(start); after each failed iteration, set the threshold to the smallest f that exceeded it. Optimal with admissible h, linear space O(bd).

Recursive Best-First Search (RBFS) — Brief

RBFS (Korf, 1993) is a recursive algorithm that imitates standard best-first search using only linear space.

- It works like depth-first search but keeps track of **f_limit**, the f-value of the best *alternative* path available from any ancestor of the current node.
- If the current node's f exceeds f_limit, the recursion **unwinds** back to the alternative path.
- While unwinding, RBFS replaces the f-value of each node along the path with a **backed-up value** — the best f-value of its children. So it remembers how good the forgotten subtree was and can decide later whether to re-expand it.

On the Romania map, RBFS first goes Arad → Sibiu → Rimnicu Vilcea. Its best child, Pitesti (417), exceeds the f_limit of 415 (Fagaras), so it unwinds and records f(Rimnicu Vilcea) = 417. It then tries Fagaras, whose best child Bucharest (450) exceeds the new limit of 417, so it unwinds and records f(Fagaras) = 450. Rimnicu Vilcea (417) is best again; this time the limit is 447 (Timisoara), so it expands Pitesti and reaches Bucharest with cost 418.

Table 2.24: Properties of RBFS

Property	RBFS
Optimal?	Yes, if h is admissible
Space	O(b·d) — linear
Time	Hard to characterise; may regenerate nodes many times ("mind changes")
Advantage over IDA*	Usually more efficient: expands in true best-first order
Weakness	Uses too little memory; cannot use extra memory even if available

Simplified Memory-Bounded A* (SMA*)

Both IDA* and RBFS use *too little* memory: IDA* keeps only the current f-limit between iterations, and RBFS keeps only the current path. If a computer has, say, enough memory for a million nodes, it is wasteful to use only a few hundred. **SMA*** (Russell, 1992) — called "small memory A*" in some syllabi — uses **all available memory**.

How SMA* Works

SMA* behaves exactly like A* until memory is full. Then:

1. **Expand the best leaf.** As in A*, the leaf with the lowest f-value is expanded (SMA* generates one successor at a time).
2. **Forget the worst leaf when memory is full.** To make room for a new node, SMA* drops the leaf with the **highest f-value** (if there is a tie, the oldest such leaf).
3. **Back up the forgotten value.** Like RBFS, the parent of a forgotten node remembers the f-value of the best forgotten child. The ancestors therefore know the quality of the best path in a forgotten subtree.
4. **Regenerate when necessary.** A forgotten subtree is regenerated only when all other paths have been shown to look worse than its remembered value.
5. **Depth limit due to memory.** If memory can hold M nodes, a node at depth M − 1 cannot have children stored. If such a node is not a goal, its f is set to ∞ (it cannot lead to a solution within memory).

SMA* Pseudocode (Outline)

```
function SMA-STAR(problem, MAX-NODES):
    OPEN <- {start}
    loop:
        if OPEN is empty: return FAILURE
        n <- deepest least-f leaf in OPEN
        if GOAL-TEST(n): return SOLUTION(n)
        s <- NEXT-SUCCESSOR(n)
        if s is not a goal and depth(s) = MAX-NODES - 1:
            f(s) <- INFINITY             // no room to go deeper
        else:
            f(s) <- max(f(n), g(s) + h(s))   // pathmax
        if all successors of n generated:
            update f(n) and its ancestors to min f of children
        if all successors of n are in memory: remove n from OPEN
        if memory is full:
            w <- shallowest highest-f leaf in OPEN
            remove w from the tree; remember f(w) in its parent
            put the parent back in OPEN if necessary
        insert s in OPEN
```

Worked Example: SMA* With Memory for 3 Nodes

Consider the search tree in Figure 2.10. Edge labels are step costs; each node shows g + h = f. D, F, I and J are goal nodes. The memory can hold only **3 nodes**, so nodes at depth 2 (like C, D, H, I) cannot have children kept in memory.

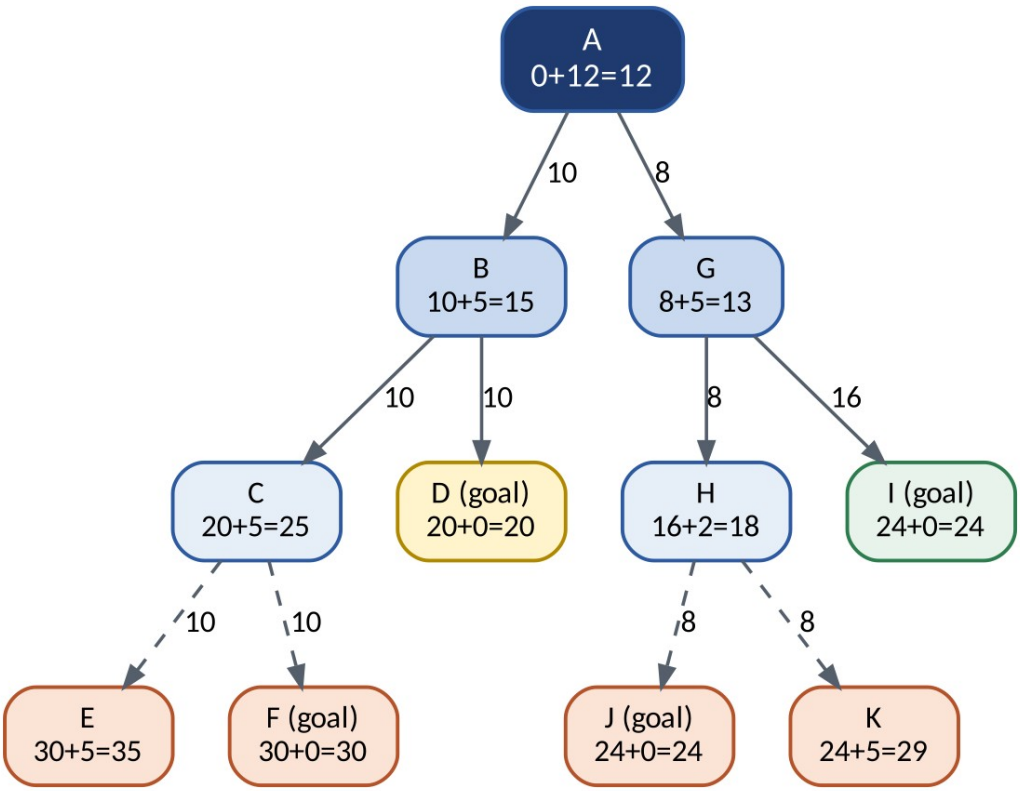


Figure 2.10: SMA example tree (memory = 3 nodes). Red nodes at depth 3 can never be stored; yellow D is the optimal goal*

Table 2.25: f-values for the SMA example (depth 0 to 2)*

Node	g	h	f = g + h	Goal?
A	0	12	12	No
B	10	5	15	No
G	8	5	13	No
C	20	5	25	No
D	20	0	20	Yes
H	16	2	18	No
I	24	0	24	Yes

The step-by-step progress of SMA* is:

Table 2.26: SMA trace with memory for 3 nodes*

Step	Action	Nodes in memory (f)	Remembered values
1	Start	A(12)	—
2	Expand A: generate B	A(12), B(15)	—
3	Generate G; memory now full; A updated to min(15, 13)	A(13), B(15), G(13)	—
4	Expand G: generate H(18); drop worst leaf B	A(13), G(13), H(18)	A remembers B = 15

Step	Action	Nodes in memory (f)	Remembered values
5	H is at max depth and not a goal, so f(H) = ∞	A(13), G(13), H(∞)	—
6	Generate I(24), a goal; drop H; G = 24	A(15), G(24), I(24)	G remembers H = ∞
7	A's best is B (15) < 24: regenerate B; memory full, drop worst leaf I	A(15), G(24), B(15)	G remembers I = 24
8	Expand B: generate C (max depth, not goal, f = ∞); drop worst leaf G	A(15), B(15), C(∞)	A remembers G = 24
9	Generate D(20), a goal; drop C; B = 20; A = min(20, 24)	A(20), B(20), D(20)	B remembers C = ∞
10	Select D (f = 20, lowest); goal test succeeds	—	Solution cost 20

At step 6, SMA* has found goal I with cost 24, but it does not stop: node A remembers that the forgotten subtree under B had f = 15 < 24, so a better solution might exist there. At step 9 it finds D with f = 20, and since 20 is now the lowest f in memory (A's other branch is remembered at 24), D is returned. **Result: A → B → D, cost 20**, which is optimal among solutions that fit in memory (and here it is the true optimum).

> **Exam Tip:** For an SMA* question, always state three rules before the trace: (1) expand the lowest-f leaf, (2) when memory is full, drop the highest-f leaf and store its f in the parent, (3) a non-goal node at maximum depth gets f = ∞. Then show the memory contents after every step, as in Table 2.26.

Properties of SMA*

*Table 2.27: Properties of SMA**

Property	SMA*
Complete?	Yes, if the shallowest goal depth d is less than the memory size (in nodes)
Optimal?	Yes, if any optimal solution is reachable within memory; otherwise returns the best reachable solution
Space	Uses exactly the memory given (M nodes)
Time	Like A* when memory is enough; can thrash when memory is very tight
Main advantage	Uses all available memory; avoids repeated states as far as memory allows
Main weakness	Thrashing — repeatedly forgetting and regenerating the same nodes

On very hard problems, SMA* can be forced to switch back and forth continually between candidate solution paths, only a few of which fit in memory. This is like **thrashing** in operating-system paging. The extra time for regeneration can make problems that A* could solve with unlimited memory intractable for SMA*. Even so, memory limits can make a problem intractable in practice, and SMA* is a sound, robust choice when memory is the binding constraint.

> **Memory Trick:** "SMA* keeps the Smartest, drops the Silliest." It always expands the best (lowest-f) leaf and, when memory is full, throws away the worst (highest-f) leaf — but the parent keeps a note of how good it was, just in case.

Comparison of All Informed Search Strategies

Table 2.28: Comparison of informed search strategies (b = branching factor, d = solution depth, m = maximum depth)

Algorithm	Evaluation / cutoff	Complete?	Optimal?	Time	Space
Greedy best-first	f = h	No (tree); yes in finite graphs	No	O(b^m)	O(b^m)
A*	f = g + h	Yes	Yes (admissible / consistent h)	Exponential	Exponential (all nodes)
AO*	Propagated cost on AND-OR graph	Yes	Yes (admissible h)	Exponential	Exponential
IDA*	DFS with f-cost threshold	Yes	Yes (admissible h)	Exponential, repeats work	O(bd)
RBFS	Recursive, f_limit and backed-up f	Yes	Yes (admissible h)	Hard to predict, regenerates	O(bd)
SMA*	A* plus forget worst leaf	Yes, if d < memory	Yes, if reachable in memory	Can thrash	Fixed (all available)

Extension: Local Search (Brief)

All the algorithms so far keep paths in memory and return a **path** to the goal. In many problems, such as the 8-queens problem, scheduling or circuit layout, only the **final state** matters, not the path. **Local search** algorithms keep only a *current state* and move to neighbouring states. They use very little memory and can find reasonable solutions in huge or continuous state spaces. Their quality measure is an **objective function** (to maximise) or a cost (to minimise).

Hill Climbing

Hill-climbing search (steepest-ascent) repeatedly moves to the neighbour with the best value, and stops when no neighbour is better.

```
function HILL-CLIMBING(problem):
    current <- initial state
    loop:
        neighbour <- highest-valued successor of current
        if VALUE(neighbour) <= VALUE(current): return current
        current <- neighbour
```

Hill climbing is often called **greedy local search**. It can get stuck at:

- **Local maxima** — a peak higher than its neighbours but lower than the global maximum;
- **Plateaus** — flat regions where all neighbours have the same value (including **shoulders**, from which progress is possible);
- **Ridges** — a sequence of local maxima that is hard to climb with single moves.

Variants include **stochastic hill climbing** (random uphill move), **first-choice hill climbing** (first better neighbour) and **random-restart hill climbing** (repeat from random starts — complete with probability approaching 1).

Simulated Annealing

Simulated annealing is inspired by annealing in metallurgy — heating a metal and cooling it slowly so that it settles into a low-energy crystalline state. The algorithm picks a *random* move. If the move improves the

state, it is always accepted. If it makes things worse by ΔE (ΔE < 0), it is accepted with probability e^(ΔE/T), where T is the **temperature**.

- At high T, bad moves are often accepted, so the search can escape local maxima.
- T is lowered gradually according to a **schedule**. As T → 0, bad moves become rare and the algorithm behaves like hill climbing.
- If T is lowered slowly enough, simulated annealing finds a global optimum with probability approaching 1.

Table 2.29: Hill climbing vs simulated annealing

Feature	Hill climbing	Simulated annealing
Move chosen	Best neighbour	Random neighbour
Accepts worse moves?	Never	Yes, with probability e^(ΔE/T)
Escapes local maxima?	No (only via restarts)	Yes, at high temperature
Memory	One state	One state
Typical use	Quick approximate optimisation	VLSI layout, scheduling, large optimisation

Summary

- A* stores all nodes; memory-bounded variants trade time for space.
- IDA* uses depth-first search with an f-cost threshold, raising it each iteration to the smallest f that was cut off. On Romania: thresholds 366, 393, 413, 415, 417, 418; solution cost 418. Space O(bd), optimal with admissible h.
- RBFS keeps an f_limit from the best alternative and backs up f-values on unwinding; linear space, optimal, but may regenerate nodes many times.
- SMA* uses all available memory, expands the best leaf, forgets the worst leaf when memory is full and stores its f in the parent. Complete if the goal depth fits in memory; optimal if an optimal solution is reachable. The example with memory 3 returns A → B → D, cost 20.
- Local search (hill climbing, simulated annealing) keeps only a current state and is used when the path does not matter.

Game Playing and Optimal (Perfect) Decisions

Games as Search Problems

So far, our agents have been alone in their environment. In **game playing**, there are two (or more) agents whose goals conflict: what is good for one player is bad for the other. This is called **adversarial search**. Games have attracted AI researchers since the very beginning of the field — Claude Shannon wrote about chess programs in 1950, and Alan Turing hand-simulated a chess program around the same time — because games are:

- **easy to formalise**: a small set of precise rules;
- **hard to solve**: chess has an average branching factor of about 35, and a game often lasts about 50 moves per player, so its search tree has roughly 35^100 ≈ 10^154 nodes (although only about 10^40 distinct positions);
- **a good test of intelligence**: they require planning, look-ahead and judgement under time limits.

Most AI game-playing work deals with **two-player, turn-taking, zero-sum games of perfect information**, such as chess, checkers, Go and tic-tac-toe. **Zero-sum** (more precisely, *constant-sum*) means that the total payoff to both players is the same in every outcome: in chess a win is 1 + 0, a draw is ½ + ½ and a loss is 0 + 1. One player's gain is exactly the other's loss.

Formal Definition of a Game

We call the two players **MAX** and **MIN**. MAX moves first, then they take turns. A game is defined by:

Table 2.30: Formal elements of a game as a search problem

Element	Meaning	Tic-tac-toe example
S0 (initial state)	Set-up at the start	Empty 3 × 3 board
PLAYER(s)	Whose turn it is in state s	X (MAX) or O (MIN)
ACTIONS(s)	Legal moves in state s	Empty squares
RESULT(s, a)	Transition model: state after move a	Board with the new mark added
TERMINAL-TEST(s)	Is the game over?	Three in a row, or board full
UTILITY(s, p)	Final numeric payoff for player p	+1 win, 0 draw, −1 loss (for X)

The utility function (also called the **objective function** or **payoff function**) is defined only on terminal states. MAX wants to reach terminal states with high utility; MIN wants low utility.

Differences From Ordinary Search

Table 2.31: Single-agent search vs adversarial search

Ordinary (single-agent) search	Game (adversarial) search
Agent controls every action	Opponent chooses half the moves
Solution is a sequence of actions (path)	Solution is a **strategy**: a move for every possible opponent reply
Goal test and path cost	Terminal test and utility (payoff)
Heuristic estimates cost to goal	Evaluation function estimates who is winning
Usually no time limit per step	Strict time limits per move

Ordinary (single-agent) search	Game (adversarial) search
Find an optimal path	Find the best move against a rational opponent

Types of Games

Games are classified along two dimensions:

1. **Deterministic vs chance (stochastic).** In deterministic games, the result of a move is fully determined (chess). In chance games, dice, card shuffles or coin tosses introduce randomness (backgammon, poker).
2. **Perfect vs imperfect information.** In games of **perfect information**, every player can see the complete state of the game at all times (chess, Go). In games of **imperfect information**, some information is hidden from at least one player (opponent's cards in bridge or poker, the hidden ships in Battleship).

Table 2.32: Classification of games

	Deterministic	Chance
Perfect information	Chess, checkers, Go, Othello, tic-tac-toe	Backgammon, Monopoly, Ludo
Imperfect information	Battleship, Kriegspiel (blind chess), Stratego	Bridge, poker, Scrabble, rummy

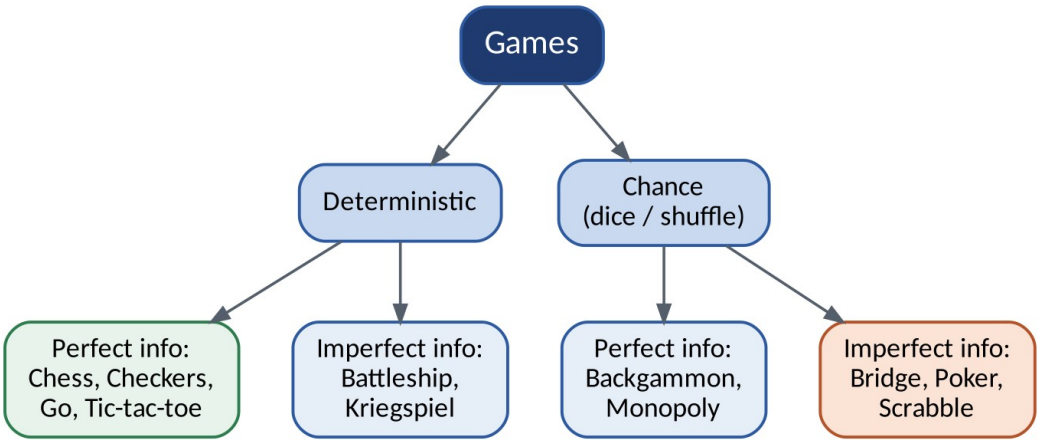


Figure 2.11: Classification of games by determinism and information

"Perfect Decision" and "Imperfect Decision" Games

The syllabus terms "perfect decision game" and "imperfect decision game" are used in two closely related ways, and you should know both:

- **Perfect (optimal) decisions** refer to choosing moves by searching the game tree all the way to the **terminal states**, using the minimax algorithm. The move found is the best possible against a perfect opponent. This is practical only for small games like tic-tac-toe, or near the end of larger games. Such perfect decisions are possible in **perfect-information** games, where the full state is known.
- **Imperfect (real-time) decisions** refer to the practical situation in chess and other large games, where there is not enough time to search to the end. The search is **cut off** at some depth and an **evaluation function** estimates how good the position is. Decisions are then only approximately optimal. Games of **imperfect information** and chance (cards, dice) also force imperfect decisions, because the true state or the outcome of moves is not fully known.

Table 2.33: Perfect decisions vs imperfect decisions in games

Aspect	Perfect decision	Imperfect decision
Search depth	To terminal states	Cut off at a depth limit
Value used at leaves	Exact utility (win/lose/draw)	Estimated by evaluation function
Quality of move	Optimal against perfect play	Approximately optimal
Time needed	O(b^m) — infeasible for chess	Fits time limit per move
Typical games	Tic-tac-toe, endgames, solved games	Chess, Go middle game; card and dice games
Algorithm	Minimax (with alpha-beta)	Depth-limited minimax / alpha-beta + EVAL

Key Point: A perfect decision is made by minimax search to the end of the game using exact utilities. An imperfect decision is made when the search must stop early (time limits) or when information is hidden or random; the leaves are then scored by an evaluation function instead of the true utility.

The Game Tree

The **game tree** is a tree in which nodes are game states and edges are moves. Levels alternate between MAX's moves and MIN's moves. A level for one player's move is called a **ply** (a "half-move"); one full move in chess is two plies.

For tic-tac-toe, MAX (X) has 9 possible first moves, MIN (O) has 8 replies, and so on. The tree has fewer than 9! = 362,880 terminal nodes, so it is small enough to search completely. Terminal states are given utilities from MAX's point of view: +1 if X wins, 0 for a draw, −1 if O wins.

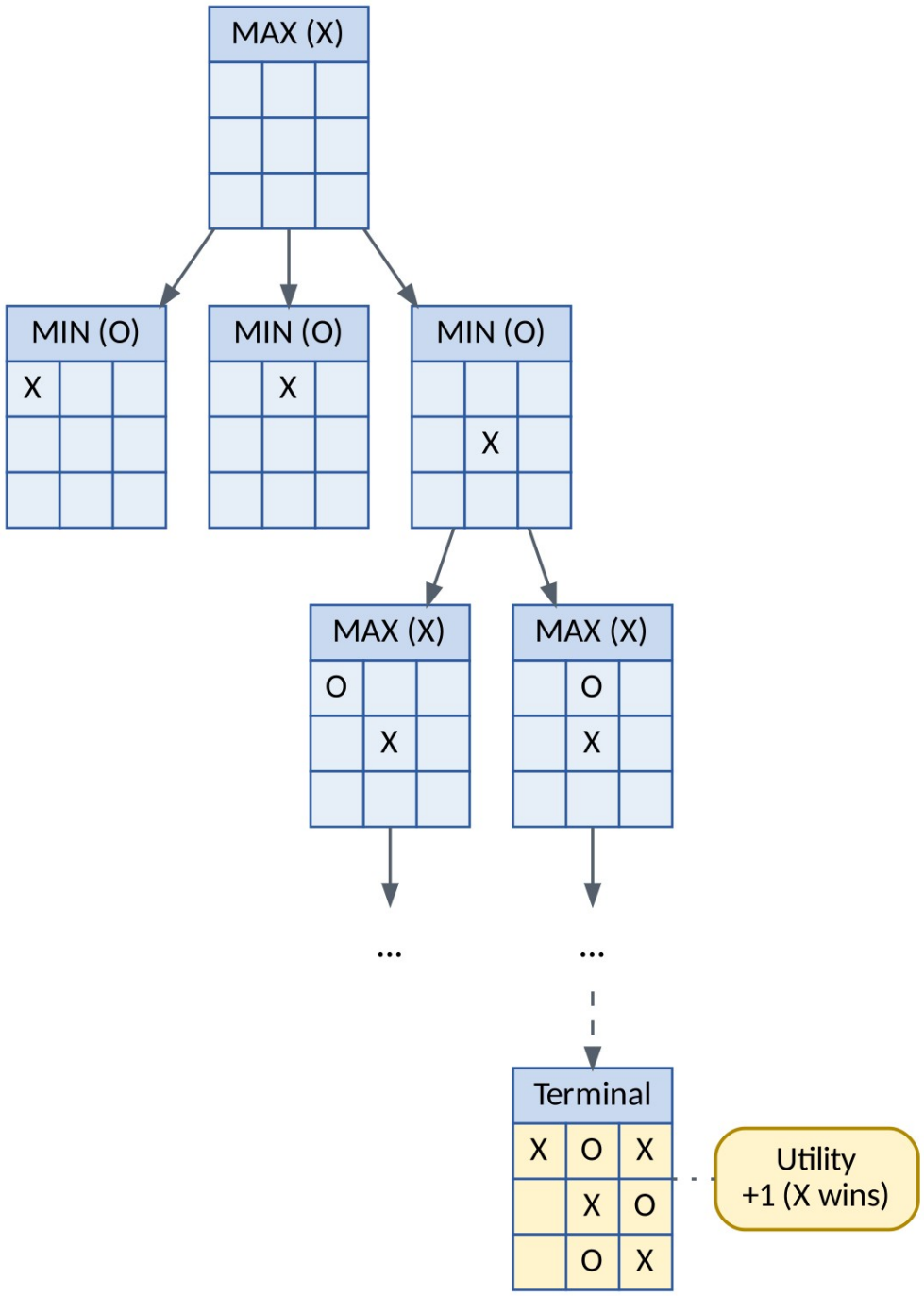


Figure 2.12: Fragment of the tic-tac-toe game tree (MAX = X, MIN = O); a terminal state with utility +1

By symmetry, only three first moves for X are genuinely different: corner, edge and centre — which is why Figure 2.12 shows just these three.

The Minimax Algorithm

Given a game tree, the optimal strategy can be found by computing the **minimax value** of each node, MINIMAX(n): the utility (for MAX) of being in that state, *assuming both players play optimally from there to the end of the game*.

```
MINIMAX(s) =
    UTILITY(s)                                      if TERMINAL-TEST(s)
    max over a in ACTIONS(s) of MINIMAX(RESULT(s, a))   if PLAYER(s) = MAX
    min over a in ACTIONS(s) of MINIMAX(RESULT(s, a))   if PLAYER(s) = MIN
```

MAX picks the move leading to the highest minimax value; MIN picks the move leading to the lowest. The values are computed bottom-up: leaves get their utilities, and values are **backed up** through the tree.

Minimax Pseudocode

```
function MINIMAX-DECISION(state):
    return the action a in ACTIONS(state)
           that maximises MIN-VALUE(RESULT(state, a))

function MAX-VALUE(state):
    if TERMINAL-TEST(state): return UTILITY(state)
    v <- -infinity
    for each a in ACTIONS(state):
        v <- MAX(v, MIN-VALUE(RESULT(state, a)))
    return v

function MIN-VALUE(state):
    if TERMINAL-TEST(state): return UTILITY(state)
    v <- +infinity
    for each a in ACTIONS(state):
        v <- MIN(v, MAX-VALUE(RESULT(state, a)))
    return v
```

The algorithm is a complete **depth-first** exploration of the game tree.

Worked Example 1: A Two-Ply Game Tree

This is the classic example from Russell and Norvig. MAX is to move at root A, with three moves a1, a2, a3 leading to MIN nodes B, C and D. The utilities of the nine terminal states (from left to right) are B: 3, 12, 8; C: 2, 4, 6; D: 14, 5, 2.

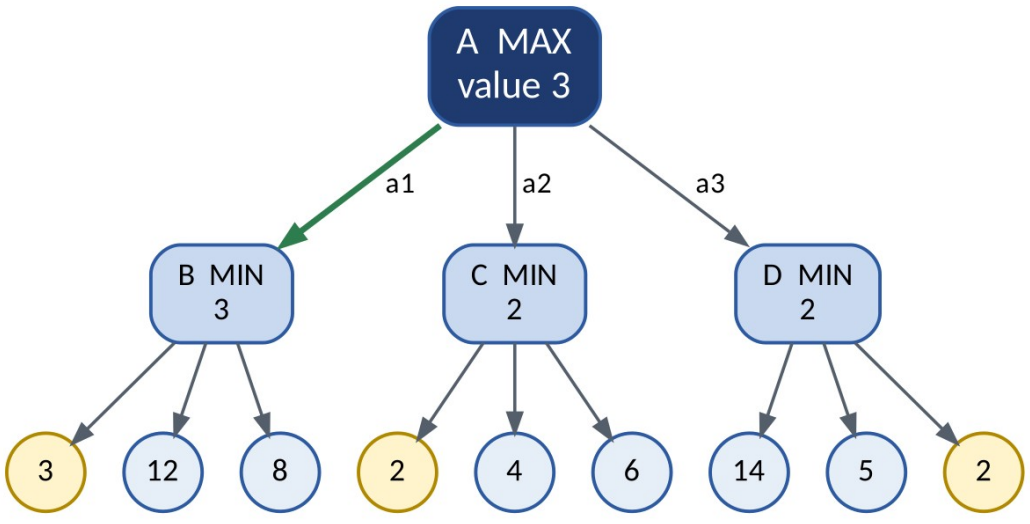


Figure 2.13: Minimax on a two-ply tree — MIN nodes back up the minimum, MAX backs up the maximum; MAX's best move is a1

```
Leaves:  B = {3, 12, 8}   C = {2, 4, 6}   D = {14, 5, 2}
MIN level:
    B = min(3, 12, 8) = 3
```

```
    C = min(2, 4, 6)  = 2
    D = min(14, 5, 2) = 2
MAX level:
    A = max(B, C, D) = max(3, 2, 2) = 3
Best move for MAX: a1 (to B); minimax value of the game = 3
```

Worked Example — Interpreting the minimax value: The value 3 means that if MAX plays a1 and both sides play optimally, MAX is guaranteed a utility of at least 3. MAX might be tempted by the 14 under D, but a rational MIN would answer a3 with the move worth 2. Minimax assumes the worst case — an optimal opponent — and picks the move whose worst outcome is best. (Values verified by a Python minimax program.)

Worked Example 2: A Three-Ply Game Tree

Root A is a MAX node with two MIN children B and C. B has MAX children D and E; C has MAX children F and G. The leaves are D: 3, 5; E: 6, 9; F: 1, 2; G: 0, −1.

Table 2.34: Minimax backed-up values for the three-ply tree

Node	Type	Children's values	Backed-up value
D	MAX	3, 5	5
E	MAX	6, 9	9
F	MAX	1, 2	2
G	MAX	0, −1	0
B	MIN	D = 5, E = 9	5
C	MIN	F = 2, G = 0	0
A	MAX	B = 5, C = 0	5

MAX should move to B, and the value of the game is 5. We shall use this same tree for alpha-beta pruning in the next chapter, where we will see that the leaf 9 and the whole subtree G need not be examined at all.

Exam Tip: In minimax questions, label every level (MAX / MIN), write the backed-up value inside or beside every internal node, and highlight the chosen move at the root. Always work bottom-up, level by level. State the final answer as "value = …, best move = …".

Properties of Minimax

Table 2.35: Properties of the minimax algorithm

Property	Minimax
Complete?	Yes, if the game tree is finite
Optimal?	Yes, against an optimal opponent
Time complexity	O(b^m), b = legal moves per position, m = maximum depth
Space complexity	O(b·m) if all successors generated at once; O(m) if one at a time
Practical limit	Chess: b ≈ 35, m ≈ 100 — exact search is impossible

What if MIN does not play optimally? Then MAX does at least as well as the minimax value — usually better. There may be other strategies that do even better against a sub-optimal opponent, but they do worse against an optimal one.

> **Memory Trick:** "MAX takes the TOP, MIN takes the BOTTOM, values bubble UP." Work from leaves upwards; at MAX levels pick the biggest number, at MIN levels the smallest.

Optimal Decisions in Multiplayer Games (Brief)

With three or more players, a single number is not enough. Each terminal state gets a **vector** of utilities, one per player — for players A, B, C it might be (1, 2, 6). At each node, the player to move chooses the child whose vector is best *for that player*. Multiplayer games often involve **alliances**, formed and broken as the game proceeds, which arise naturally from optimal strategies for each player.

Summary

- Games are adversarial search problems defined by initial state, PLAYER, ACTIONS, RESULT, TERMINAL-TEST and UTILITY.
- Games are classified as deterministic or chance, and as perfect or imperfect information (chess vs backgammon vs bridge).
- A perfect decision searches to terminal states with exact utilities (minimax); an imperfect decision cuts off the search and uses an evaluation function, and is also forced by hidden information or chance.
- Minimax backs up values: MAX nodes take the maximum, MIN nodes the minimum. In the classic tree (3, 12, 8 / 2, 4, 6 / 14, 5, 2) the value is 3 and MAX plays a1.
- Minimax is complete and optimal but needs time O(b^m) — far too slow for chess, which motivates cutoffs and alpha-beta pruning.

Imperfect Decisions, Evaluation Functions and Alpha-Beta Pruning

Imperfect Real-Time Decisions

Minimax searches all the way down to terminal states. For tic-tac-toe this is fine, but for chess (b ≈ 35, m ≈ 100) it is hopeless: even at a billion positions per second, the full tree would take longer than the age of the universe. Real games also impose a **time limit per move** — for example, a few minutes in tournament chess.

In 1950, Claude Shannon proposed the standard solution in his paper *Programming a Computer for Playing Chess*: cut off the search early and apply a **heuristic evaluation function** to the positions at the cutoff, treating them as if they were terminal. This turns non-terminal nodes into "pseudo-leaves". The resulting decisions are **imperfect** — they may not be optimal — but they can be made in real time.

Two changes are made to minimax:

1. Replace the **TERMINAL-TEST** with a **CUTOFF-TEST** that decides when to stop searching.
2. Replace the **UTILITY** function with a heuristic **evaluation function EVAL** that estimates the expected utility of a position.

```
H-MINIMAX(s, d) =
    EVAL(s)                                      if CUTOFF-TEST(s, d)
    max over a of H-MINIMAX(RESULT(s, a), d + 1)   if PLAYER(s) = MAX
    min over a of H-MINIMAX(RESULT(s, a), d + 1)   if PLAYER(s) = MIN
```

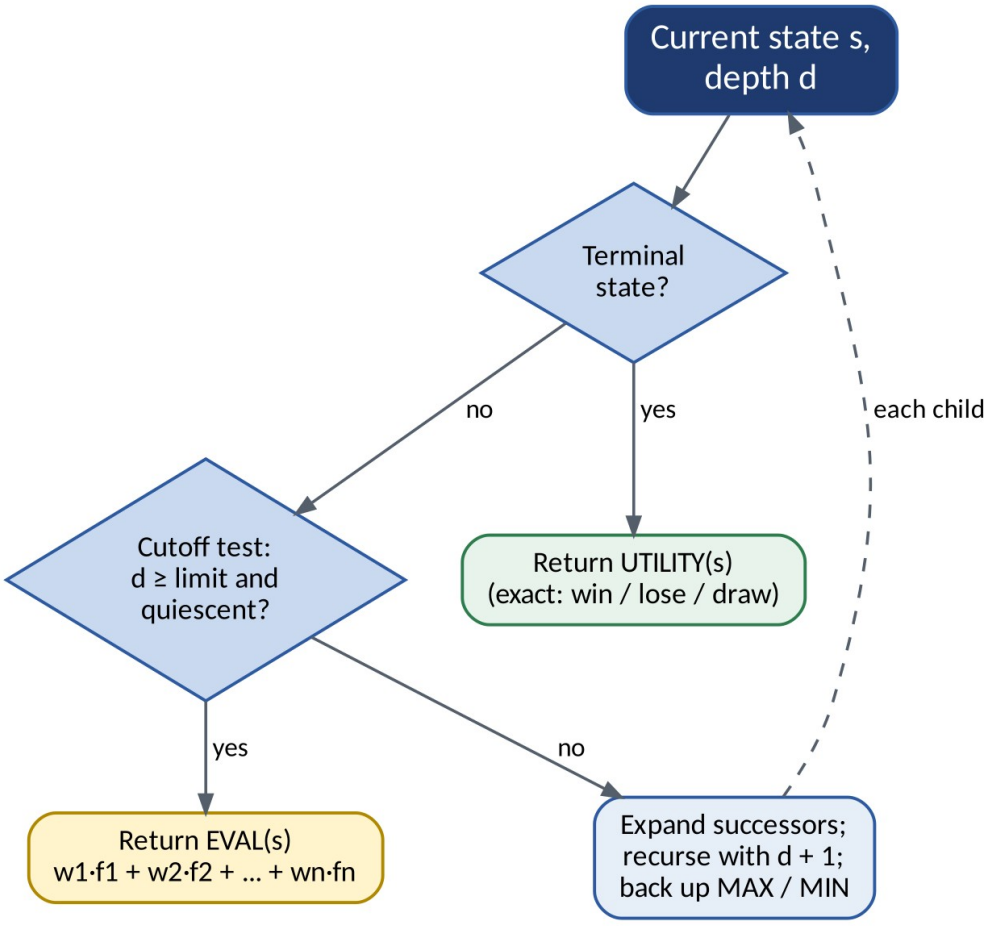


Figure 2.14: Flow of depth-limited (heuristic) minimax with cutoff test and evaluation function

The Cutoff Test

The simplest cutoff test is a **fixed depth limit**: CUTOFF-TEST(s, d) is true if d is greater than some fixed depth, or if s is terminal. Better approaches:

- **Iterative deepening:** search to depth 1, then 2, then 3, … and when time runs out, play the move chosen by the deepest completed search. This guarantees a move is always available, and the results of shallower searches help order moves for deeper ones.
- **Quiescence-based cutoff:** only stop at positions that are quiet (see below).
- **Singular extensions:** search deeper along a move that is clearly better than all others.

The Evaluation Function

An **evaluation function** EVAL(s) returns an estimate of the expected utility of the game from position s, from MAX's point of view. It plays the same role for games that the heuristic h(n) plays in single-agent search: it guesses how good a state is without searching to the end.

Requirements of a Good Evaluation Function

1. **Ordering of terminal states:** it should rank terminal states in the same order as the true utility function (wins above draws above losses).
2. **Fast to compute:** it is evaluated at every cutoff node, so it must be cheap.
3. **Strongly correlated with the chance of winning** for non-terminal states.

Weighted Linear Evaluation Functions

Most evaluation functions compute numerical **features** of a position and combine them as a **weighted linear function**:

EVAL(s) = w1·f1(s) + w2·f2(s) + … + wn·fn(s)

where each fi is a feature (for example, the number of white queens minus black queens) and each wi is a weight expressing its importance.

Evaluation Function for Chess: Material Value

Chess books give approximate **material values** for each piece:

Table 2.36: Standard chess material values

Piece	Material value
Pawn	1
Knight	3
Bishop	3
Rook	5
Queen	9
King	Not counted (cannot be captured)

A simple evaluation is **material balance** = (sum of White's piece values) − (sum of Black's piece values). Other features used in practice include pawn structure, king safety, mobility (number of legal moves), centre control and a "good pawn structure" bonus. For example, two bishops together are often worth a little more than twice one bishop.

Material evaluation in chess. White has a queen, two rooks and five pawns: 9 + 5 + 5 + 5 × 1 = 24. Black has a queen, a rook, a bishop, a knight and six pawns: 9 + 5 + 3 + 3 + 6 × 1 = 26. EVAL = 24 − 26 = −2 from White's point of view, so the position is estimated to favour Black by about two pawns. A material-only evaluation ignores everything else, which is why real programs add many more features.

Evaluation Function for Tic-Tac-Toe

A classical evaluation function for tic-tac-toe (from Nilsson and from Rich and Knight) is:

E(n) = M(n) − O(n)

where

- **M(n)** = number of lines (rows, columns, diagonals) still **open for MAX (X)** — lines that contain no O;
- **O(n)** = number of lines still **open for MIN (O)** — lines that contain no X.

If n is a win for MAX, E(n) = +∞; if a win for MIN, E(n) = −∞. There are 8 lines in total.

Table 2.37: Tic-tac-toe evaluation E(n) = M(n) − O(n) for sample positions (verified by program)

Position (X = MAX, O = MIN)	M(n)	O(n)	E(n) = M − O
X in centre only	8	4	4
X in a corner only	8	5	3
X on an edge only	8	6	2
X centre, O in a corner	5	4	1
X centre, O on an edge	6	4	2

Position (X = MAX, O = MIN)	M(n)	O(n)	E(n) = M − O
X in a corner, O in centre	4	5	−1

Worked Example — Two-ply search with E(n) for X's first move: Search X's three distinct first moves, then all of O's replies, and score each leaf with E(n). X in centre: O's best reply (a corner) gives E = 1, so the backed-up value is 1. X in a corner: O's best reply (the centre) gives E = −1. X on an edge: O's best reply (the centre) gives E = −2. MAX takes max(1, −1, −2) = 1, so X should open in the centre. (All values computed by a Python program.)

Key Point: An evaluation function EVAL(s) estimates the expected utility of a non-terminal position. It is usually a weighted linear sum of features, w1·f1 + … + wn·fn — material balance in chess, open lines M − O in tic-tac-toe. It must order terminal states correctly, be fast, and correlate with winning chances.

Limitations of Linear Evaluation Functions

A linear function assumes each feature contributes independently. In reality features interact — a bishop is more valuable in the endgame, and two bishops together are worth more than the sum of their parts. Modern programs therefore use non-linear combinations, and since the 2010s deep neural networks trained on millions of positions, as in AlphaGo and its successors.

The Horizon Effect

The **horizon effect** occurs when the program faces an unavoidable loss (or damage) that lies just beyond its search depth, and it makes delaying moves that push the loss over the "horizon" where it cannot be seen. The bad event is not prevented, only postponed — and the delaying moves often make things worse.

Example. Suppose Black's bishop is trapped and will be captured by White in a few moves. A depth-limited Black program may keep giving check with a pawn or sacrificing pawns, each sacrifice pushing the capture of the bishop beyond the depth limit. The program "thinks" it has saved the bishop, but it has merely lost several pawns *and* will still lose the bishop.

Remedies for the horizon effect:

- **Singular extensions:** if one move is clearly better than all others in a position, search it more deeply even beyond the normal depth limit.
- **Quiescence search** (below).
- Deeper search, made possible by alpha-beta pruning and fast hardware.

Quiescence Search

The evaluation function should be applied only to positions that are **quiescent** — positions unlikely to show wild swings in value in the near future. A position in the middle of a queen exchange, where White has just captured a queen and Black is about to recapture, is not quiescent: a material count at that moment would wildly overestimate White's position.

Quiescence search is an extra search, beyond the cutoff depth, that expands only non-quiescent positions — typically only **capture moves** (and sometimes checks) — until a quiet position is reached. Only then is EVAL applied. This avoids evaluating "half-finished" exchanges and reduces errors caused by the fixed depth cutoff.

Table 2.38: Problems of cutoff search and their remedies

Problem	Cause	Remedy
Wrong value in mid-exchange	EVAL applied to a non-quiet position	Quiescence search (expand captures only)
Horizon effect	Inevitable loss pushed beyond depth limit	Singular extensions, quiescence search
Not enough depth in time limit	b^d nodes too many	Alpha-beta pruning, move ordering, iterative deepening
Repeated positions	Different move orders reach same state	Transposition table (hash table of evaluated positions)

Exam Tip: Questions on "imperfect real-time decisions" usually expect four terms: cutoff test, evaluation function (with chess material values or tic-tac-toe M − O example), horizon effect (with the delaying-moves explanation) and quiescence search. Write a short paragraph on each and add the H-MINIMAX formula.

Alpha-Beta Pruning

The Idea

Minimax examines every node of the game tree. But often we can **compute the correct minimax decision without looking at every node**. **Pruning** means eliminating parts of the tree that cannot possibly influence the final decision. **Alpha-beta pruning** returns *exactly the same move* as minimax, but prunes away branches that cannot affect the result.

Look again at the two-ply tree in Figure 2.13. After seeing B = 3, MAX knows the root is worth at least 3. At C, the first leaf is 2, so MIN at C can force a value of at most 2. Since 2 < 3, MAX will never choose C, whatever the other leaves under C are. So the remaining leaves of C (4 and 6) need not be examined. Formally:

MINIMAX(root) = max(min(3, 12, 8), min(2, x, y), min(14, 5, 2)) = max(3, min(2, x, y), 2) = max(3, z, 2) where z ≤ 2 = 3.

The answer does not depend on x and y.

Alpha and Beta

The algorithm carries two bounds down the tree:

- **α (alpha)** = the value of the best (highest) choice found so far for **MAX** along the path to the root. MAX is guaranteed at least α. α starts at −∞ and only increases.
- **β (beta)** = the value of the best (lowest) choice found so far for **MIN** along the path to the root. MIN is guaranteed at most β. β starts at +∞ and only decreases.

Pruning rule: whenever α ≥ β at a node, the remaining children of that node can be pruned.

- **β-cutoff** (at a MAX node): the MAX node's value v ≥ β. MIN above it already has a better option (≤ β), so MIN will never let the game reach here.
- **α-cutoff** (at a MIN node): the MIN node's value v ≤ α. MAX above it already has a better option (≥ α), so MAX will never come here.

Alpha-Beta Pseudocode

```
function ALPHA-BETA-SEARCH(state):
    v <- MAX-VALUE(state, -infinity, +infinity)
    return the action in ACTIONS(state) with value v

function MAX-VALUE(state, alpha, beta):
    if TERMINAL-TEST(state): return UTILITY(state)
    v <- -infinity
    for each a in ACTIONS(state):
        v <- MAX(v, MIN-VALUE(RESULT(state, a), alpha, beta))
        if v >= beta: return v          // beta cutoff: prune rest
        alpha <- MAX(alpha, v)
    return v

function MIN-VALUE(state, alpha, beta):
    if TERMINAL-TEST(state): return UTILITY(state)
    v <- +infinity
    for each a in ACTIONS(state):
        v <- MIN(v, MAX-VALUE(RESULT(state, a), alpha, beta))
        if v <= alpha: return v         // alpha cutoff: prune rest
        beta <- MIN(beta, v)
    return v
```

For a depth-limited version, replace TERMINAL-TEST by CUTOFF-TEST and UTILITY by EVAL.

Worked Example 1: The Two-Ply Tree (Figure 2.13)

Table 2.39: Alpha-beta trace on the two-ply tree

Step	Node	Event	α, β after step	Value so far
1	B (MIN)	Leaf 3	B: α = −∞, β = 3	B ≤ 3
2	B	Leaf 12: no change	β = 3	B ≤ 3
3	B	Leaf 8: no change; B = 3	—	B = 3
4	A (MAX)	Receives 3	A: α = 3, β = +∞	A ≥ 3
5	C (MIN)	Leaf 2: v = 2 ≤ α = 3	α-cutoff	C ≤ 2
6	C	Leaves 4 and 6 pruned	—	C = 2 (bound)
7	D (MIN)	Leaf 14	D: α = 3, β = 14	D ≤ 14
8	D	Leaf 5	β = 5	D ≤ 5
9	D	Leaf 2: D = 2	—	D = 2
10	A	max(3, 2, 2) = 3	—	A = 3, move a1

Only 7 of the 9 leaves were examined; the leaves 4 and 6 under C were pruned. D could not be pruned because its smallest leaf (2) came last. If D's leaves had been ordered 2, 5, 14, the first leaf 2 ≤ α = 3 would have pruned the other two leaves as well — a first glimpse of the importance of **move ordering**.

Worked Example 2: Step-by-Step on a Three-Ply Tree

We now use the three-ply tree from the previous chapter: A (MAX) has MIN children B and C; B has MAX children D (leaves 3, 5) and E (leaves 6, 9); C has MAX children F (leaves 1, 2) and G (leaves 0, −1). Children are searched from left to right.

```
MAX-VALUE(A, α=-∞, β=+∞)
 └ MIN-VALUE(B, α=-∞, β=+∞)
    ├ MAX-VALUE(D, α=-∞, β=+∞)
```

```
    │   leaf 3 → v = 3, α = 3
    │   leaf 5 → v = 5, α = 5          D returns 5
    │ B: v = 5, β = 5
    ├ MAX-VALUE(E, α=-∞, β=5)
    │   leaf 6 → v = 6;  v ≥ β (6 ≥ 5) → β-CUTOFF, prune leaf 9
    │                                   E returns 6
    │ B: v = min(5, 6) = 5               B returns 5
A: v = 5, α = 5
└ MIN-VALUE(C, α=5, β=+∞)
    ├ MAX-VALUE(F, α=5, β=+∞)
    │   leaf 1 → v = 1
    │   leaf 2 → v = 2                  F returns 2
    │ C: v = 2;  v ≤ α (2 ≤ 5) → α-CUTOFF, prune subtree G
    │                                   C returns 2
A: v = max(5, 2) = 5                    Best move: A → B
```

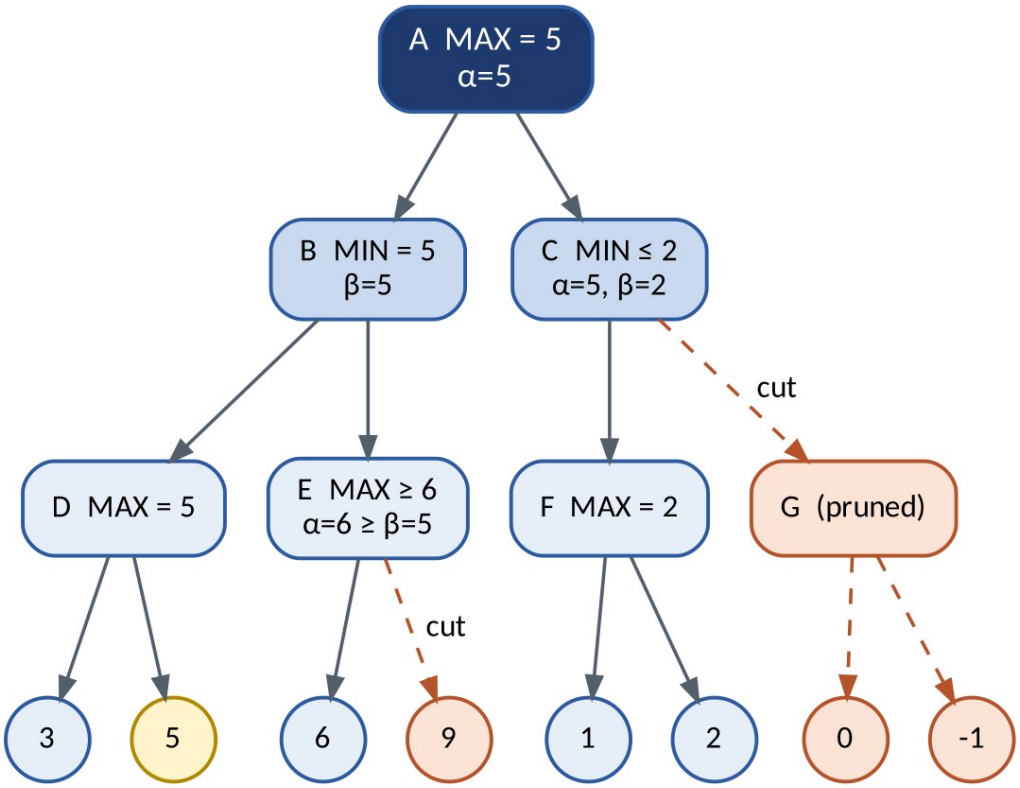


Figure 2.15: Alpha-beta pruning on a three-ply tree — leaf 9 (β-cutoff at E) and the whole subtree G (α-cutoff at C) are pruned; root value 5

Table 2.40: Branches pruned in the three-ply example

Pruned part	Where	Type	Reason
Leaf 9 under E	MAX node E	β-cutoff	E ≥ 6 already, but MIN at B can get 5 via D; B will never choose E
Subtree G (leaves 0, −1)	MIN node C	α-cutoff	C ≤ 2 already, but MAX at A can get 5 via B; A will never choose C

Result: root value **5**, best move **A → B** — exactly the same as plain minimax (Table 2.34), but only 5 of the 8 leaves were evaluated. (Both alpha-beta examples were verified by a Python program that logs every cutoff.)

Why E's value is reported as 6, not 9. Alpha-beta returned 6 for E, but the true minimax value of E is 9. This is fine: once E is known to be at least 6 and B can already get 5, the exact value of E is irrelevant to B's decision. Alpha-beta guarantees the correct value at the root, not at pruned interior nodes — a subtle point that examiners sometimes test.

> **Memory Trick:** "Alpha is MAX's floor, Beta is MIN's ceiling. When the floor meets the ceiling (α ≥ β), the room collapses — prune!" Also: MAX updates α, MIN updates β.

Move Ordering and Complexity

The effectiveness of alpha-beta depends heavily on the **order in which successors are examined**.

Table 2.41: Effect of move ordering on alpha-beta pruning

Ordering of moves	Nodes examined (approx.)	Effective branching factor	Depth searchable in same time
Worst case (worst moves first)	O(b^m) — no pruning	b	Same as minimax
Random order	O(b^(3m/4))	b^(3/4)	About 4/3 of minimax depth
Perfect ordering (best moves first)	O(b^(m/2))	√b	Twice the minimax depth

With perfect ordering, the effective branching factor drops from b to √b. For chess, that is about 6 instead of 35, so alpha-beta can look ahead roughly **twice as far** as plain minimax in the same time.

Perfect ordering is impossible in practice (if we knew the best move we would not need to search), but simple ideas come close:

- **Try captures first**, then threats, then forward moves, then backward moves.
- **Killer-move heuristic:** try first the moves that were best (caused cutoffs) in sibling positions.
- **Iterative deepening:** use the best moves from the depth d − 1 search to order moves at depth d.
- **Transposition tables:** store values of positions already evaluated (a hash table), since the same position can arise from different move orders.

Table 2.42: Minimax vs alpha-beta pruning

Feature	Minimax	Alpha-beta pruning
Result at root	Optimal minimax value	Same value and same move
Nodes examined	All nodes, O(b^m)	Best case O(b^(m/2)), worst O(b^m)
Extra parameters	None	α and β bounds
Depends on move order?	No	Yes, strongly
Space	O(bm)	O(bm)

> **Key Point:** Alpha-beta pruning never changes the minimax decision; it only skips branches that cannot influence it. With perfect move ordering it examines O(b^(m/2)) nodes instead of O(b^m), effectively doubling the search depth.

Games With Chance: Expectiminimax (Brief)

In **stochastic games** such as backgammon, a dice roll decides which moves are legal. The game tree must therefore include **chance nodes** between the MAX and MIN levels. Each branch from a chance node is labelled with a possible outcome (e.g. a dice roll) and its probability. With two dice in backgammon there

are 36 outcomes but only 21 distinct rolls (doubles have probability 1/36; the other 15 rolls have probability 1/18 each).

The value of a chance node is the **expected value** — the probability-weighted average of its children:

```
EXPECTIMINIMAX(s) =
    UTILITY(s)                                        if TERMINAL-TEST(s)
    max over a of EXPECTIMINIMAX(RESULT(s, a))         if PLAYER(s) = MAX
    min over a of EXPECTIMINIMAX(RESULT(s, a))         if PLAYER(s) = MIN
    Σ over r of P(r) · EXPECTIMINIMAX(RESULT(s, r))   if PLAYER(s) = CHANCE
```

Small example. MAX chooses between move L and move R, each followed by a fair coin toss. After L, the outcomes are worth 2 and 4: expected value 0.5 × 2 + 0.5 × 4 = 3. After R, the outcomes are worth 0 and 5: expected value 0.5 × 0 + 0.5 × 5 = 2.5. MAX chooses L (value 3), even though R contains the single best outcome.

Points to note:

- Time complexity becomes O(b^m · n^m), where n is the number of distinct chance outcomes — much larger than minimax.
- The **scale** of the evaluation function now matters: only a positive linear transformation of EVAL preserves the decisions, because averages are taken. (In deterministic games, any order-preserving transformation is fine.)
- Alpha-beta style pruning is possible only if bounds on the utility values are known.

For **card games** (imperfect information), a common approach is to consider many possible deals of the hidden cards, solve each as a perfect-information game, and choose the move that is best on average — though this "averaging over clairvoyance" can be misleading because it ignores the value of gathering information.

State-of-the-Art Game Programs (Brief)

Table 2.43: Landmark game-playing programs

Year	Program	Game	Achievement
1994	Chinook	Checkers	Became official world man-machine champion; checkers was later weakly solved (2007) — perfect play leads to a draw
1997	Deep Blue (IBM)	Chess	Defeated world champion Garry Kasparov in a six-game match, 3.5–2.5
1997	Logistello	Othello	Defeated world champion Takeshi Murakami 6–0
2016	AlphaGo (DeepMind)	Go	Defeated Lee Sedol, one of the world's strongest players, 4–1
2017	AlphaZero (DeepMind)	Chess, shogi, Go	Learned purely by self-play and beat leading programs in each game

- **Deep Blue** was a parallel computer with special-purpose chess chips. It used alpha-beta search with iterative deepening, singular extensions, a large opening book and endgame databases, and

searched more than 100 million positions per second. Its evaluation function used several thousand features.

- **AlphaGo** faced Go's much larger branching factor (about 250 on a 19 × 19 board), which defeated traditional alpha-beta programs. It combined **Monte Carlo tree search** with **deep neural networks**: a *policy network* to suggest promising moves and a *value network* to evaluate positions, trained on expert games and then by self-play reinforcement learning.

These programs show the progression of ideas in this unit: from exact minimax, to depth-limited search with hand-crafted evaluation functions and alpha-beta pruning (Deep Blue), to learned evaluation functions guiding sampled search (AlphaGo).

Summary

- Real games need **imperfect real-time decisions**: replace TERMINAL-TEST with CUTOFF-TEST and UTILITY with EVAL (H-MINIMAX).
- Evaluation functions are usually weighted linear sums of features: material values in chess (P 1, N 3, B 3, R 5, Q 9); E(n) = M(n) − O(n) in tic-tac-toe, which makes the centre X's best first move (two-ply value 1).
- The horizon effect pushes unavoidable losses beyond the search depth; quiescence search and singular extensions reduce it.
- Alpha-beta pruning keeps α (best for MAX) and β (best for MIN) and prunes when α ≥ β. On the three-ply tree, leaf 9 and subtree G are pruned and the root value remains 5.
- With perfect ordering alpha-beta needs O(b^(m/2)) time, doubling the searchable depth; random ordering gives about O(b^(3m/4)).
- Expectiminimax handles chance nodes by expected values. Deep Blue (1997) and AlphaGo (2016) are landmarks of game-playing AI.

Unit 2 Cheat Sheet

Core Definitions in One Line

- **Informed search:** search that uses a problem-specific heuristic h(n) to choose which node to expand.
- **Heuristic h(n):** estimated cost of the cheapest path from n to a goal; h(goal) = 0.
- **h*(n):** true cost of the cheapest path from n to a goal.
- **g(n):** actual path cost from the start to n.
- **Best-first search:** expand the OPEN node with the lowest evaluation f(n).
- **Greedy best-first:** f(n) = h(n).
- **A*:** f(n) = g(n) + h(n) = estimated cost of cheapest solution through n.
- **Admissible:** h(n) ≤ h*(n) for all n (never overestimates).
- **Consistent / monotonic:** h(n) ≤ c(n, a, n′) + h(n′); consistent ⇒ admissible.
- **Dominance:** h2 dominates h1 if h2(n) ≥ h1(n) for all n (both admissible).
- **Effective branching factor b*:** N + 1 = 1 + b* + (b*)^2 + … + (b*)^d.
- **IDA*:** DFS with f-cost threshold; next threshold = smallest f that exceeded it.
- **RBFS:** recursive best-first with f_limit and backed-up f-values; linear space.
- **SMA*:** A* that uses all memory; drops highest-f leaf when full; parent remembers its f.
- **Minimax value:** utility for MAX assuming optimal play by both sides.
- **Evaluation function:** estimate of expected utility at a cutoff; EVAL = Σ wi·fi.
- **Horizon effect:** unavoidable loss pushed beyond the search depth by delaying moves.
- **Quiescence search:** extend search at non-quiet positions (captures) before applying EVAL.
- **Alpha-beta:** α = best for MAX so far, β = best for MIN so far; prune when α ≥ β.

Algorithms in One Line Each

Table 2.44: Unit 2 algorithms in one line

Algorithm	One-line description
Greedy	Pop lowest h from OPEN; expand; stop when goal popped
A*	Pop lowest g + h; goal-test on pop; update g if a cheaper path appears
IDA*	T = h(start); repeat DFS cutting off f > T; T = min cut-off f
RBFS	Recurse into best child with limit min(f_limit, second-best f); back up f on return
SMA*	A* until memory full; drop worst leaf; store its f in parent; f = ∞ at max depth
Minimax	Leaves = utility; MAX node = max of children; MIN node = min of children
H-minimax	Same as minimax but EVAL at CUTOFF-TEST instead of UTILITY at terminal
Alpha-beta	MAX: if v ≥ β return; α = max(α, v). MIN: if v ≤ α return; β = min(β, v)

Algorithm	One-line description
Expectiminimax	Chance node value = Σ P(r) × value of child r

Key Numbers to Remember

Table 2.45: Numbers and results worth memorising

Item	Value
Romania greedy path (Arad→Sibiu→Fagaras→Bucharest)	450
Romania A* optimal path (Arad→Sibiu→RV→Pitesti→Bucharest)	418
A* f-values along optimal path	366, 393, 413, 417, 418
IDA* thresholds on Romania	366, 393, 413, 415, 417, 418
8-puzzle start 724/5_6/831: h1, h2, h*	8, 18, 26
8-puzzle reachable states	9!/2 = 181,440
SMA* example (memory 3)	Returns A→B→D, cost 20
Minimax tree 3,12,8 / 2,4,6 / 14,5,2	Value 3, move a1; alpha-beta prunes 4 and 6
Three-ply tree 3,5 / 6,9 / 1,2 / 0,−1	Value 5; prunes leaf 9 and subtree G
Chess material	P 1, N 3, B 3, R 5, Q 9
Alpha-beta best case	O(b^(m/2)); random O(b^(3m/4))
Minimax time / space	O(b^m) / O(bm)

Comparison Tables

Table 2.46: Informed searches at a glance

Algorithm	Complete	Optimal	Space
Greedy	No (tree)	No	O(b^m)
A*	Yes	Yes (admissible / consistent)	Exponential
IDA*	Yes	Yes	O(bd)
RBFS	Yes	Yes	O(bd)
SMA*	If d < memory	If reachable in memory	Fixed M

Table 2.47: Game classification at a glance

Game type	Deterministic	Chance
Perfect information	Chess, Go, tic-tac-toe	Backgammon, Monopoly
Imperfect information	Battleship, Kriegspiel	Poker, bridge, Scrabble

Table 2.48: Perfect vs imperfect decisions at a glance

Perfect decision	Imperfect decision
Search to terminal states	Cut off at depth limit

Perfect decision	Imperfect decision
Exact UTILITY	Estimated EVAL
Optimal, but O(b^m)	Approximate, real-time

Heuristic Facts

- h1 = misplaced tiles; h2 = Σ Manhattan distances; never count the blank.
- h2 ≥ h1 always, so h2 dominates and expands no more nodes.
- max(h1, ..., hm) of admissible heuristics is admissible and dominates each.
- Relaxed problem cost = admissible and consistent heuristic.
- Relax "adjacent and blank": remove both → h1; remove "blank" → h2.
- Learned heuristic: h(n) = c1·x1(n) + c2·x2(n); not guaranteed admissible.

Game-Playing Facts

- MAX moves first; ply = one player's move.
- Alpha-beta gives the same root value as minimax.
- β-cutoff happens at a MAX node (v ≥ β); α-cutoff at a MIN node (v ≤ α).
- Good move ordering: captures first, killer moves, iterative deepening, transposition tables.
- Tic-tac-toe E(n) = open lines for X − open lines for O; centre opening gives E = 4.
- Deep Blue beat Kasparov 1997 (alpha-beta + special hardware); AlphaGo beat Lee Sedol 4–1 in 2016 (MCTS + deep neural networks).

Memory Tricks

- **"G, H, G-plus-H"** — f = g gives Uniform-cost, f = h gives Greedy, f = g + h gives A*.
- **"OPEN = To-Do, CLOSED = Done."**
- **"Admissible = Always optimistic; Consistent = Climbs steadily."**
- **"Test the goal when you POP, not when you PUSH"** — the key A* rule.
- **"IDA* raises the bar to the lowest jump that failed."**
- **"SMA* keeps the Smartest, drops the Silliest."**
- **"Relax the rules, get a heuristic."**
- **"MAX takes the TOP, MIN takes the BOTTOM, values bubble UP."**
- **"Alpha is MAX's floor, Beta is MIN's ceiling; floor meets ceiling → prune."**
- **"CHEQ" for imperfect decisions** — Cutoff test, Horizon effect, Evaluation function, Quiescence search.

Important Questions

Two-Mark Questions (With Model Answers)

Q1. What is informed search? Informed (heuristic) search uses problem-specific knowledge, in the form of a heuristic function h(n), to decide which node to expand next. Examples are greedy best-first search and A*.

Q2. Define a heuristic function. A heuristic function h(n) is the estimated cost of the cheapest path from the state at node n to a goal state. It is cheap to compute and h(goal) = 0. Example: straight-line distance to the destination.

Q3. What is best-first search? Best-first search is a graph-search strategy that always expands the node on OPEN with the lowest value of an evaluation function f(n). With f = h it becomes greedy search; with f = g + h it becomes A*.

Q4. Why is greedy best-first search not optimal? It uses only h(n) and ignores g(n), the cost already paid. On the Romania map it goes Arad → Sibiu → Fagaras → Bucharest (450 km) instead of the optimal 418 km route through Rimnicu Vilcea and Pitesti.

Q5. Write the evaluation function of A* and explain each term. f(n) = g(n) + h(n), where g(n) is the actual cost from the start to n, h(n) is the estimated cost from n to the goal, and f(n) is the estimated total cost of the cheapest solution through n.

Q6. What is an admissible heuristic? A heuristic is admissible if it never overestimates the true cost to the goal: h(n) ≤ h*(n) for every node n. Straight-line distance is admissible for route finding.

Q7. Define a consistent (monotonic) heuristic. h is consistent if for every node n and successor n′, h(n) ≤ c(n, a, n′) + h(n′), and h(goal) = 0. Every consistent heuristic is admissible, and f-values never decrease along a path.

Q8. What is meant by dominance of heuristics? If h1 and h2 are admissible and h2(n) ≥ h1(n) for all n, h2 dominates h1. A* with h2 never expands more nodes than with h1; e.g. Manhattan distance dominates misplaced tiles.

Q9. Define effective branching factor. If A* generates N nodes for a solution at depth d, the effective branching factor b* satisfies N + 1 = 1 + b* + (b*)^2 + … + (b*)^d. Values close to 1 indicate a good heuristic.

Q10. What is IDA*? Iterative Deepening A* is a depth-first search that cuts off nodes whose f = g + h exceeds a threshold. After each iteration the threshold becomes the smallest f-value that was cut off. It is optimal with admissible h and uses O(bd) space.

Q11. What is SMA*? Simplified (small) Memory-bounded A* behaves like A* until memory is full, then drops the leaf with the highest f-value and stores that value in its parent. It is complete if the goal depth fits in memory.

Q12. What is a game tree? Define ply. A game tree has game states as nodes and moves as edges, with levels alternating between MAX and MIN. A ply is one move by one player (a half-move); one full chess move is two plies.

Q13. What is an evaluation function in game playing? EVAL(s) estimates the expected utility of a non-terminal position from MAX's point of view, used when search is cut off. It is usually a weighted linear sum, w1·f1(s) + … + wn·fn(s), such as material balance in chess.

Q14. What is the horizon effect? The horizon effect occurs when a depth-limited program uses delaying moves to push an unavoidable loss beyond its search depth (horizon). The loss is not avoided, only hidden; quiescence search and singular extensions reduce it.

Q15. What do α and β represent in alpha-beta pruning? α is the best (highest) value found so far for MAX along the path; β is the best (lowest) value found so far for MIN. A branch is pruned when α ≥ β.

Five-Mark Questions (With Answer Outlines)

Q1. Differentiate between informed and uninformed search strategies. Outline: definition of each; table of at least six differences (knowledge used, node selection rule, efficiency, cost per node, optimality conditions, examples) — see Table 2.3; one-line example: BFS vs A*; conclude that a good heuristic reduces nodes by orders of magnitude (e.g. 3.6 million vs 73 nodes for the 8-puzzle at depth 12).

Q2. Explain greedy best-first search with an example. State its properties. Outline: f(n) = h(n); best-first algorithm with OPEN/CLOSED; Romania trace Arad → Sibiu → Fagaras → Bucharest (Table 2.7); cost 450 vs optimal 418; properties: not optimal, incomplete as tree search (Iasi–Neamt loop), time and space O(b^m); advantage: fast with a good heuristic.

Q3. Explain admissible and consistent heuristics. Show that consistency implies admissibility. Outline: definitions with formulas; straight-line distance as example (Table 2.11); triangle-inequality diagram (Figure 2.7); proof by induction: for a node one step from the goal, h(n) ≤ c + 0 = h*(n); assume true for nodes k steps away; then h(n) ≤ c + h(n′) ≤ c + h*(n′) = h*(n); consequences: f non-decreasing, no reopening of CLOSED nodes; comparison table (Table 2.12).

Q4. Explain the 8-puzzle heuristics h1 and h2 with an example. Which is better and why? Outline: describe the puzzle; define h1 (misplaced tiles) and h2 (Manhattan distance); compute both for start 7 2 4 / 5 _ 6 / 8 3 1 → h1 = 8, h2 = 18 (Table 2.16); both admissible with reasons; h2 dominates h1; experimental data (Table 2.18: 227 vs 73 nodes, b* 1.42 vs 1.24); relaxed-problem origin of both.

Q5. Explain IDA* with an example. Compare it with A*. Outline: motivation (A* memory); algorithm steps and pseudocode; threshold update rule; Romania trace thresholds 366 → 393 → 413 → 415 → 417 → 418 (Table 2.22); properties: complete, optimal, O(bd) space, repeated work; comparison: A* stores all nodes, IDA* stores only the path; IDA* weak with real-valued costs.

Q6. Explain SMA* algorithm. What are its advantages and limitations? Outline: motivation (IDA* and RBFS use too little memory); rules: expand best leaf, drop worst leaf when full, back up forgotten f into parent, f = ∞ for non-goal at max depth, regenerate when needed; small trace (Table 2.26); properties: complete if d < memory, optimal if reachable; limitation: thrashing.

Q7. Explain imperfect real-time decisions in game playing. Outline: why minimax to terminal states is infeasible (chess b ≈ 35, m ≈ 100); H-MINIMAX formula; cutoff test (fixed depth, iterative deepening); evaluation function (weighted linear; chess material values; tic-tac-toe M − O); horizon effect with example; quiescence search; flowchart (Figure 2.14).

Q8. Explain the minimax algorithm with a suitable example. Outline: definition of MAX and MIN, utility; minimax value formula; pseudocode (MAX-VALUE, MIN-VALUE); worked tree 3, 12, 8 / 2, 4, 6 / 14, 5, 2 → B = 3, C = 2, D = 2, root = 3, move a1 (Figure 2.13); properties: complete, optimal, time O(b^m), space O(bm).

Ten-Mark Questions (With Structured Answer Plans)

Q1 (Numerical). Apply A* to the graph below and find the optimal path from S to G. Show OPEN and CLOSED lists at every step.

Directed edges with costs: S→A 1, S→B 4, A→B 2, A→C 5, A→D 12, B→C 2, C→D 3, C→G 7, D→G 3. Heuristic: h(S) = 7, h(A) = 6, h(B) = 4, h(C) = 2, h(D) = 1, h(G) = 0.

Full solution (verified by running A* in Python).

Table 2.49: A solution for ten-mark Q1*

Step	Expand (g, f)	Successors generated: g + h = f	OPEN after step	CLOSED
0	—	—	S(7)	—
1	S (0, 7)	A: 1 + 6 = 7; B: 4 + 4 = 8	A(7), B(8)	S
2	A (1, 7)	B: 3 + 4 = 7 (better, replaces 8); C: 6 + 2 = 8; D: 13 + 1 = 14	B(7), C(8), D(14)	S, A
3	B (3, 7)	C: 5 + 2 = 7 (better, replaces 8)	C(7), D(14)	S, A, B
4	C (5, 7)	D: 8 + 1 = 9 (better, replaces 14); G: 12 + 0 = 12	D(9), G(12)	S, A, B, C
5	D (8, 9)	G: 11 + 0 = 11 (better, replaces 12)	G(11)	S, A, B, C, D
6	G (11, 11)	Goal test succeeds	—	all

Answer: Optimal path S → A → B → C → D → G, cost 1 + 2 + 2 + 3 + 3 = **11**.

Answer plan for writing: (1) state f = g + h and the goal-on-pop rule (1 mark); (2) check admissibility — true costs h*(D) = 3, h*(C) = 6, h*(B) = 8, h*(A) = 10, h*(S) = 11, all ≥ h (2 marks); (3) the table above (5 marks); (4) final path and cost, noting that G was first generated with f = 12 but A* continued and found f = 11 (2 marks).

Q2 (Numerical). Apply minimax and alpha-beta pruning to the tree below. Show α and β values and identify the pruned branches.

Root A is MAX with three MIN children B, C, D. B has MAX children E (leaves 2, 3) and F (leaves 5, 9). C has MAX children G (leaves 0, 1) and H (leaves 7, 4). D has MAX children I (leaves 6, 8) and J (leaves 2, 1).

Full solution (verified in Python).

Minimax values: E = 3, F = 9, G = 1, H = 7, I = 8, J = 2; B = min(3, 9) = 3; C = min(1, 7) = 1; D = min(8, 2) = 2; A = max(3, 1, 2) = **3**; best move A → B.

Alpha-beta trace:

```
A (MAX) α=-∞ β=+∞
 B (MIN) α=-∞ β=+∞
   E (MAX): leaf 2 → v=2, α=2; leaf 3 → v=3       E = 3
   B: v=3, β=3
   F (MAX) α=-∞ β=3: leaf 5 → v=5 ≥ β=3 → CUT (leaf 9 pruned)
   B = min(3, 5) = 3
 A: v=3, α=3
 C (MIN) α=3 β=+∞
```

```
  G (MAX): leaf 0 → v=0; leaf 1 → v=1          G = 1
  C: v=1 ≤ α=3 → CUT (subtree H with 7, 4 pruned)
A: still 3
D (MIN) α=3 β=+∞
  I (MAX): leaf 6, leaf 8                      I = 8;  D: β=8
  J (MAX) α=3 β=8: leaf 2 → v=2; leaf 1 → v=2   J = 2
  D: v=2 ≤ α=3 (no children left to prune)     D = 2
A = max(3, 1, 2) = 3
```

Answer: Root value 3, move A → B. Pruned: leaf 9 (β-cutoff at F) and subtree H (leaves 7 and 4, α-cutoff at C). Nine of twelve leaves were evaluated.

Answer plan: (1) define α, β and the pruning rule (2 marks); (2) minimax values at all nodes (2 marks); (3) step-wise trace with α, β (4 marks); (4) list of pruned branches with type of cutoff and final answer (2 marks).

Q3. Explain the A* algorithm in detail. Prove that A* is optimal when the heuristic is admissible. Illustrate with the Romania example.

Answer plan:

1. Introduction: best-first search, f = g + h, meaning of each term (1 mark).
2. Pseudocode with OPEN/CLOSED and path update (2 marks).
3. Romania trace: Table 2.9 and Figure 2.6, emphasising Bucharest generated at 450 but selected at 418 (3 marks).
4. Admissibility and consistency definitions (1 mark).
5. Optimality proof: suppose suboptimal goal G2 on OPEN with f(G2) = g(G2) > C*; some node n on the optimal path is on OPEN with f(n) ≤ C*; so n is expanded before G2 (2 marks).
6. Properties table (complete, optimal, optimally efficient, exponential space) and conclusion on memory problem (1 mark).

Q4. What are memory-bounded heuristic search algorithms? Explain IDA* and SMA* in detail and compare all informed search strategies.

Answer plan:

1. Motivation: A* keeps all nodes; memory is the bottleneck (1 mark).
2. IDA*: algorithm, threshold rule, pseudocode, Romania iterations (Table 2.22, Figure 2.9), properties (3 marks).
3. RBFS: one paragraph on f_limit and backed-up values (1 mark).
4. SMA*: rules, example with memory 3 (Figure 2.10, Table 2.26), properties and thrashing (3 marks).
5. Comparison table of greedy, A*, IDA*, RBFS, SMA* (Table 2.28) (2 marks).

MCQ Practice

Choose the single best answer for each question. The answer key with one-line explanations follows at the end.

1. Informed search differs from uninformed search because it uses:

- (a) a queue instead of a stack
- (b) problem-specific knowledge in a heuristic function
- (c) a fixed depth limit
- (d) random choice of successors

2. In best-first search, the node selected for expansion is the one with:

- (a) the greatest depth
- (b) the lowest evaluation function f(n)
- (c) the highest heuristic value
- (d) the earliest generation time

3. Greedy best-first search uses the evaluation function:

- (a) f(n) = g(n)
- (b) f(n) = g(n) + h(n)
- (c) f(n) = h(n)
- (d) f(n) = depth(n)

4. On the Romania map, greedy search from Arad reaches Bucharest with path cost:

- (a) 418
- (b) 450
- (c) 366
- (d) 493

5. In A*, f(n) represents:

- (a) the exact cost from start to n
- (b) the estimated cost from n to goal
- (c) the estimated cost of the cheapest solution through n
- (d) the number of nodes expanded

6. A heuristic h is admissible if:

- (a) $h(n) \geq h^*(n)$ for all n
- (b) $h(n) \leq h^*(n)$ for all n
- (c) $h(n) = 0$ for all n
- (d) $h(n) \leq c(n, a, n')$ for all n

7. The condition $h(n) \leq c(n, a, n') + h(n')$ defines a:

- (a) dominant heuristic
- (b) relaxed heuristic
- (c) consistent heuristic
- (d) learned heuristic

8. Which statement is TRUE?

- (a) Every admissible heuristic is consistent
- (b) Every consistent heuristic is admissible
- (c) Consistent heuristics may overestimate
- (d) Admissibility and consistency are unrelated

9. In A*, the goal test is applied when a node is:

- (a) generated
- (b) inserted in CLOSED
- (c) selected (removed) from OPEN for expansion

- (d) found to have h = 0

10. The A* path from Arad to Bucharest in the Romania subset passes through:

- (a) Fagaras
- (b) Timisoara and Lugoj
- (c) Rimnicu Vilcea and Pitesti
- (d) Zerind and Oradea

11. If h(n) = 0 for all n, A* behaves like:

- (a) depth-first search
- (b) greedy best-first search
- (c) uniform-cost search
- (d) hill climbing

12. The major drawback of A* is its:

- (a) lack of optimality
- (b) exponential memory requirement
- (c) incompleteness
- (d) inability to use heuristics

13. For the 8-puzzle start 7 2 4 / 5 _ 6 / 8 3 1 with goal _ 1 2 / 3 4 5 / 6 7 8, the Manhattan distance h2 is:

- (a) 8
- (b) 14
- (c) 18
- (d) 26

14. h2 (Manhattan) dominates h1 (misplaced tiles) because:

- (a) h2 is easier to compute

(b) h2(n) ≥ h1(n) for every n and both are admissible

- (c) h2 overestimates
- (d) h1 is not admissible

15. If A* generates 52 nodes to reach a solution at depth 5, the effective branching factor is about:

- (a) 1.24
- (b) 1.92
- (c) 2.78
- (d) 10.4

16. Removing all restrictions on tile movement in the 8-puzzle gives the heuristic:

- (a) Manhattan distance
- (b) Gaschnig's heuristic
- (c) number of misplaced tiles
- (d) pattern database

17. IDA* uses as its cutoff:

- (a) depth of the node
- (b) f-cost = g + h
- (c) number of nodes in memory
- (d) the heuristic h alone

18. After an unsuccessful IDA* iteration, the new threshold is:

- (a) the old threshold + 1
- (b) double the old threshold
- (c) the smallest f-value that exceeded the old threshold
- (d) the largest f-value seen

19. The space complexity of IDA* is:

- (a) O(b^d)
- (b) O(bd)
- (c) O(d^b)
- (d) O(1)

20. When memory is full, SMA* drops:

- (a) the oldest node
- (b) the root
- (c) the leaf with the lowest f-value
- (d) the leaf with the highest f-value

21. SMA* is complete if:

- (a) memory is infinite only
- (b) the depth of the shallowest goal is less than the memory size
- (c) the heuristic is zero
- (d) the branching factor is 2

22. In a game tree, one ply means:

- (a) one complete game
- (b) one move by one player
- (c) one move by both players
- (d) one level of chance nodes

23. Backgammon is an example of a game that is:

- (a) deterministic, perfect information
- (b) chance, perfect information
- (c) deterministic, imperfect information

- (d) chance, imperfect information

24. In the minimax tree with MIN nodes having leaves (3, 12, 8), (2, 4, 6), (14, 5, 2), the root value is:

- (a) 2
- (b) 3
- (c) 12
- (d) 14

25. The time complexity of minimax is:

- (a) O(bm)
- (b) O(b^m)
- (c) O(b^(m/2))
- (d) O(m^b)

26. In heuristic minimax for real-time decisions, the TERMINAL-TEST is replaced by:

- (a) GOAL-TEST
- (b) CUTOFF-TEST
- (c) UTILITY
- (d) RESULT

27. In the tic-tac-toe evaluation function E(n) = M(n) − O(n), a board with only X in the centre has E equal to:

- (a) 2
- (b) 3
- (c) 4
- (d) 8

28. The horizon effect is reduced by:

- (a) breadth-first search

- (b) quiescence search and singular extensions
- (c) removing the evaluation function
- (d) reducing the depth limit

29. In alpha-beta pruning, a branch is pruned when:

- (a) $\alpha < \beta$
- (b) $\alpha \geq \beta$
- (c) $\alpha = 0$
- (d) $\beta = +\infty$

30. With perfect move ordering, alpha-beta examines about:

- (a) O(b^m) nodes
- (b) O(b^(3m/4)) nodes
- (c) O(b^(m/2)) nodes
- (d) O(m) nodes

Answer Key

Table 2.50: MCQ answer key

Q	Answer	Explanation
1	(b)	Informed search uses a heuristic built from problem knowledge
2	(b)	Best-first always expands the lowest-f node on OPEN
3	(c)	Greedy search ignores g and uses only h
4	(b)	Arad–Sibiu–Fagaras–Bucharest = 140 + 99 + 211 = 450
5	(c)	f = g + h estimates the total cost of a solution through n
6	(b)	Admissible means never overestimating the true cost h^*
7	(c)	This triangle inequality is the definition of consistency

Q	Answer	Explanation
8	(b)	Consistency implies admissibility, not the reverse
9	(c)	Testing on selection is what guarantees optimality
10	(c)	Optimal path Arad–Sibiu–Rimnicu Vilcea–Pitesti–Bucharest, cost 418
11	(c)	With h = 0, f = g, which is uniform-cost search
12	(b)	A* stores all generated nodes, so memory grows exponentially
13	(c)	3 + 1 + 2 + 2 + 2 + 3 + 3 + 2 = 18
14	(b)	Higher admissible values everywhere means dominance
15	(b)	Solving 1 + b + … + b^5 = 53 gives b* ≈ 1.92
16	(c)	Tiles can jump anywhere, so each misplaced tile needs one move
17	(b)	IDA* deepens on f-cost contours, not depth
18	(c)	The threshold rises to the smallest f that was cut off
19	(b)	Only the current path and its siblings are stored
20	(d)	SMA* forgets the worst (highest-f) leaf and backs up its value
21	(b)	The solution path must fit in the available memory
22	(b)	A ply is a half-move, one player's turn
23	(b)	Dice add chance, but the board is fully visible
24	(b)	MIN values 3, 2, 2; MAX picks 3
25	(b)	Minimax explores the full tree of depth m
26	(b)	Search stops at a cutoff and applies EVAL
27	(c)	M = 8 open lines for X, O = 4 open lines for O; 8 − 4 = 4
28	(b)	Searching deeper at non-quiet or forced positions reveals the hidden loss
29	(b)	When α ≥ β the node cannot affect the decision
30	(c)	Effective branching factor becomes √b, doubling search depth

Glossary

Table 2.51: Glossary of Unit 2 terms

Term	Meaning
A* search	Best-first search with f(n) = g(n) + h(n); optimal with an admissible heuristic
Admissible heuristic	A heuristic that never overestimates: h(n) ≤ h*(n)
Adversarial search	Search in environments with opponents whose goals conflict, as in games
Alpha (α)	Best value found so far for MAX along the current path; a lower bound
Alpha-beta pruning	Minimax variant that skips branches which cannot affect the decision
AND-OR graph	Graph where a node may need all children solved (AND) or any one (OR)
AO*	Best-first search algorithm for AND-OR graphs with cost propagation
Beta (β)	Best value found so far for MIN along the current path; an upper bound
Best-first search	Strategy that expands the node with the lowest evaluation f(n)
Chance node	Node in a stochastic game tree representing a random event such as a dice roll
CLOSED list	Set of nodes already expanded (explored set)
Consistent heuristic	Heuristic satisfying h(n) ≤ c(n, a, n′) + h(n′); also called monotonic
Cutoff test	Test that decides when to stop searching and apply the evaluation function
Dominance	h2 dominates h1 if h2(n) ≥ h1(n) for all n, both admissible
Effective branching factor	b* with N + 1 = 1 + b* + … + (b*)^d; measures heuristic quality
Evaluation function (games)	EVAL(s): estimate of expected utility of a non-terminal position
Expectiminimax	Minimax extended with chance nodes valued by expected value
f-contour	Region of states with f(n) ≤ some value; A* expands nodes contour by contour
g(n)	Actual cost of the path from the start node to node n
Game tree	Tree of game states with alternating MAX and MIN levels
Greedy best-first search	Best-first search with f(n) = h(n)
h(n)	Heuristic estimate of the cheapest cost from n to a goal
h*(n)	True cost of the cheapest path from n to a goal

Term	Meaning
Hill climbing	Local search that always moves to the best neighbouring state
Horizon effect	Postponing an unavoidable loss beyond the search depth by delaying moves
IDA*	Iterative Deepening A*: DFS with an increasing f-cost threshold
Manhattan distance	Sum of horizontal and vertical distances; heuristic h2 for the 8-puzzle
Material value	Numeric worth of chess pieces: pawn 1, knight 3, bishop 3, rook 5, queen 9
Minimax	Algorithm that backs up max values at MAX nodes and min values at MIN nodes
Misplaced tiles	Count of tiles not in their goal square; heuristic h1 for the 8-puzzle
Move ordering	Examining likely best moves first to increase alpha-beta pruning
OPEN list	Priority queue of generated but unexpanded nodes (the frontier)
Optimally efficient	No other optimal algorithm with the same heuristic expands fewer nodes (A*)
Pattern database	Stored exact costs of subproblems used as an admissible heuristic
Perfect information	Every player can see the complete game state (chess, Go)
Ply	One move by one player; a half-move
Quiescence search	Extra search of non-quiet positions (e.g. captures) before applying EVAL
RBFS	Recursive Best-First Search: linear-space best-first with backed-up f-values
Relaxed problem	Problem with fewer restrictions; its optimal cost is an admissible heuristic
Simulated annealing	Local search that accepts worse moves with probability e^(ΔE/T)
SMA*	Simplified Memory-bounded A*: uses all memory and drops the worst leaf when full
Thrashing	Repeated forgetting and regenerating of nodes in SMA* under tight memory
Utility function	Numeric payoff of a terminal game state for a player
Zero-sum game	Game in which total payoff to all players is the same in every outcome